Storage

Ways to provide both long-term and temporary storage to Pods in your cluster.

1 - Volumes

Kubernetes *volumes* provide a way for containers in a Pod to access and share data via the filesystem. There are different kinds of volume that you can use for different purposes, such as:

- populating a configuration file based on a ConfigMap or a Secret
- providing some temporary scratch space for a Pod
- sharing a filesystem between two different containers in the same Pod
- sharing a filesystem between two different Pods (even if those Pods run on different nodes)
- durably storing data so that it stays available even if the Pod restarts or is replaced

- passing configuration information to an app running in a container, based on details of the Pod the container is in (for example: telling a sidecar container what namespace the Pod is running in)
- providing read-only access to data in a different container image

Data sharing can be between different local processes within a container, or between different containers, or between Pods.

Why volumes are important

- **Data persistence:** On-disk files in a container are ephemeral, which presents some problems for non-trivial applications when running in containers. One problem occurs when a container crashes or is stopped; the container state is not saved, so all of the files that were created or modified during the lifetime of the container are lost. After a crash, kubelet restarts the container with a clean state.

- **Shared storage:** Another problem occurs when multiple containers are running in a `Pod` and need to share files. It can be challenging to set up and access a shared filesystem across all of the containers.

The Kubernetes volume abstraction can help you to solve both of these problems.

Before you learn about volumes, PersistentVolumes, and PersistentVolumeClaims, you should read up about Pods and make sure that you understand how Kubernetes uses Pods to run containers.

How volumes work

Kubernetes supports many types of volumes. A Pod can use any number of volume types simultaneously. Ephemeral volume types have a lifetime linked to a specific Pod, but persistent volumes exist beyond the lifetime of any individual Pod. When a Pod ceases to exist, Kubernetes destroys ephemeral volumes; however, Kubernetes does not destroy persistent volumes. For any kind of volume in a given Pod, data is preserved across container restarts.

At its core, a volume is a directory, possibly with some data in it, which is accessible to the containers in a pod. How that directory comes to be, the medium that backs it, and the contents of it are determined by the particular volume type used.

To use a volume, specify the volumes to provide for the Pod in `.spec.volumes` and declare where to mount those volumes into containers in `.spec.containers[*].volumeMounts`.

When a Pod is launched, a process in the container sees a filesystem view composed from the initial contents of the container image, plus volumes (if defined) mounted inside the container. The process sees a root filesystem that initially matches the contents of the container image. Any writes to within that filesystem hierarchy, if allowed, affect what that process views when it performs a subsequent filesystem access. Volumes are mounted at specified paths within the container filesystem. For each container defined within a Pod, you must independently specify where to mount each volume that the container uses.

Volumes cannot mount within other volumes (but see Using subPath for a related mechanism). Also, a volume cannot contain a hard link to anything in a different volume.

Types of volumes

Kubernetes supports several types of volumes.

configMap

A ConfigMap provides a way to inject configuration data into Pods. The data stored in a ConfigMap can be referenced in a volume of type `configMap` and then consumed by containerized applications running in a Pod.

When referencing a ConfigMap, you provide the name of the ConfigMap in the volume. You can customize the path to use for a specific entry in the ConfigMap. The following configuration shows how to mount the `log-config` ConfigMap onto a Pod called `configmap-pod`：

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
    - name: test
      image: busybox:1.28
      command: ['sh', '-c', 'echo "The app is running!" && tail -f /dev/null']
      volumeMounts:
        - name: config-vol
```

```
          mountPath: /etc/config
  volumes:
    - name: config-vol
      configMap:
        name: log-config
        items:
          - key: log_level
            path: log_level.conf
```

The `log-config` ConfigMap is mounted as a volume, and all contents stored in its `log_level` entry are mounted into the Pod at path `/etc/config/log_level.conf`. Note that this path is derived from the volume's `mountPath` and the `path` keyed with `log_level`.

> **Note:**
> - You must create a ConfigMap before you can use it.
> - A ConfigMap is always mounted as `readOnly`.
> - A container using a ConfigMap as a `subPath` volume mount will not receive updates when the ConfigMap changes.
> - Text data is exposed as files using the UTF-8 character encoding. For other character encodings, use `binaryData`.

downwardAPI

A `downwardAPI` volume makes downward API data available to applications. Within the volume, you can find the exposed data as read-only files in plain text format.

> **Note:**
> A container using the downward API as a `subPath` volume mount does not receive updates when field values change.

See Expose Pod Information to Containers Through Files to learn more.

emptyDir

For a Pod that defines an `emptyDir` volume, the volume is created when the Pod is assigned to a node. As the name says, the `emptyDir` volume is initially empty. All containers in the Pod can read and write the same files in the `emptyDir` volume, though that volume can be mounted at the same or different paths in each container. When a Pod is removed from a node for any reason, the data in the `emptyDir` is deleted permanently.

> **Note:**
> A container crashing does *not* remove a Pod from a node. The data in an `emptyDir` volume is safe across container crashes.

Some uses for an `emptyDir` are:

- scratch space, such as for a disk-based merge sort
- checkpointing a long computation for recovery from crashes
- holding files that a content-manager container fetches while a webserver container serves the data

The `emptyDir.medium` field controls where `emptyDir` volumes are stored. By default `emptyDir` volumes are stored on whatever medium that backs the node such as disk, SSD, or network storage, depending on your environment. If you set the `emptyDir.medium` field to `"Memory"`, Kubernetes mounts a tmpfs (RAM-backed filesystem) for you instead. While tmpfs is very fast, be aware that, unlike disks, files you write count against the memory limit of the container that wrote them.

A size limit can be specified for the default medium, which limits the capacity of the `emptyDir` volume. The storage is allocated from node ephemeral storage. If that is filled up from another source (for example, log files or image overlays), the `emptyDir` may run out of capacity before this limit. If no size is specified, memory-backed volumes are sized to node allocatable memory.

> **Caution:**
> Please check here for points to note in terms of resource management when using memory-backed `emptyDir`.

emptyDir configuration example

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir:
      sizeLimit: 500Mi
```

emptyDir memory configuration example

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir:
      sizeLimit: 500Mi
      medium: Memory
```

fc (fibre channel)

An `fc` volume type allows an existing fibre channel block storage volume to be mounted in a Pod. You can specify single or multiple target world wide names (WWNs) using the

parameter `targetWWNs` in your Volume configuration. If multiple WWNs are specified, targetWWNs expect that those WWNs are from multi-path connections.

> **Note:**
> You must configure FC SAN Zoning to allocate and mask those LUNs (volumes) to the target WWNs beforehand so that Kubernetes hosts can access them.

gcePersistentDisk (deprecated)

In Kubernetes 1.36, all operations for the in-tree `gcePersistentDisk` type are redirected to the `pd.csi.storage.gke.io` CSI driver.

The `gcePersistentDisk` in-tree storage driver was deprecated in the Kubernetes v1.17 release and then removed entirely in the v1.28 release.

The Kubernetes project suggests that you use the Google Compute Engine Persistent Disk CSI third party storage driver instead.

gitRepo (disabled)

> **Warning:**
>
> Kubernetes 1.36 does *not* include the `gitRepo` volume driver. The last version that provided a way to use this driver was Kubernetes v1.35, and it has been deprecated since the v1.11 minor release.
>
> To provision a Pod that has a Git repository mounted, you can mount an `emptyDir` volume into an init container that clones the repo using Git, then mount the EmptyDir into the Pod's container.
>
> ---
>
> You can restrict the use of `gitRepo` volumes in your cluster using policies, such as ValidatingAdmissionPolicy. You can use the following Common Expression Language (CEL) expression as part of a policy to reject use of `gitRepo` volumes:
>
> ```
> !has(object.spec.volumes) || !object.spec.volumes.exists(v, has(v.gitRepo))
> ```

hostPath

A `hostPath` volume mounts a file or directory from the host node's filesystem into your Pod. This is not something that most Pods will need, but it offers a powerful escape hatch for some applications.

> **Warning:**
>
> Using the `hostPath` volume type presents many security risks. If you can avoid using a `hostPath` volume, you should. For example, define a `local` PersistentVolume, and use that instead.
>
> If you are restricting access to specific directories on the node using admission-time validation, that restriction is only effective when you additionally require that any mounts of that `hostPath` volume are **read only**. If you allow a read-write mount of any host path by an untrusted Pod, the containers in that Pod may be able to subvert the read-write host mount.
>
> ---
>
> Take care when using `hostPath` volumes, whether these are mounted as read-only or as read-write, because:
>
> - Access to the host filesystem can expose privileged system credentials (such as for the kubelet) or privileged APIs (such as the container runtime socket) that can be used for container escape or to attack other parts of the cluster.
> - Pods with identical configuration (such as created from a PodTemplate) may behave differently on different nodes due to different files on the nodes.
> - `hostPath` volume usage is not treated as ephemeral storage usage. You need to monitor the disk usage by yourself because excessive `hostPath` disk usage will lead to disk pressure on the node.

Some uses for a `hostPath` are:

- running a container that needs access to node-level system components (such as a container that transfers system logs to a central location, accessing those logs using a read-only mount of `/var/log`)
- making a configuration file stored on the host system available read-only to a static Pod; unlike normal Pods, static Pods cannot access ConfigMaps

`hostPath` volume types

In addition to the required `path` property, you can optionally specify a `type` for a `hostPath` volume.

The available values for `type` are:

Value	Behavior
`""`	Empty string (default) is for backward compatibility, which means that no checks will be performed before mounting the `hostPath` volume.
`DirectoryOrCreate`	If nothing exists at the given path, an empty directory will be created there as needed with permission set to 0755, having the same group and ownership with Kubelet.
`Directory`	A directory must exist at the given path.
`FileOrCreate`	If nothing exists at the given path, an empty file will be created there as needed with permission set to 0644, having the same group and ownership with Kubelet.
`File`	A file must exist at the given path.
`Socket`	A UNIX socket must exist at the given path.
`CharDevice`	*(Linux nodes only)* A character device must exist at the given path.
`BlockDevice`	*(Linux nodes only)* A block device must exist at the given path.

> **Caution:**
>
> The `FileOrCreate` mode does **not** create the parent directory of the file. If the parent directory of the mounted file does not exist, the Pod fails to start. To ensure that this mode works, you can try to mount directories and files separately, as shown in the `FileOrCreate` example for `hostPath`.

Some files or directories created on the underlying hosts might only be accessible by root. You then either need to run your process as root in a privileged container or modify the file permissions on the host to read from or write to a `hostPath` volume.

hostPath configuration example

Linux node | Windows node

```yaml
---
# This manifest mounts /data/foo on the host as /foo inside the
# single container that runs within the hostpath-example-linux Pod.
#
# The mount into the container is read-only.
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-example-linux
spec:
  os: { name: linux }
  nodeSelector:
    kubernetes.io/os: linux
  containers:
  - name: example-container
    image: registry.k8s.io/test-webserver
    volumeMounts:
    - mountPath: /foo
      name: example-volume
      readOnly: true
  volumes:
  - name: example-volume
    # mount /data/foo, but only if that directory already exists
    hostPath:
      path: /data/foo # directory location on host
      type: Directory # this field is optional
```

hostPath FileOrCreate configuration example

The following manifest defines a Pod that mounts `/var/local/aaa` inside the single container in the Pod. If the node does not already have a path `/var/local/aaa`, the kubelet creates it as a directory and then mounts it into the Pod.

If `/var/local/aaa` already exists but is not a directory, the Pod fails. Additionally, the kubelet attempts to make a file named `/var/local/aaa/1.txt` inside that directory (as seen

from the host); if something already exists at that path and isn't a regular file, the Pod fails.

Here's the example manifest:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-webserver
spec:
  os: { name: linux }
  nodeSelector:
    kubernetes.io/os: linux
  containers:
  - name: test-webserver
    image: registry.k8s.io/test-webserver:latest
    volumeMounts:
    - mountPath: /var/local/aaa
      name: mydir
    - mountPath: /var/local/aaa/1.txt
      name: myfile
  volumes:
  - name: mydir
    hostPath:
      # Ensure the file directory is created.
      path: /var/local/aaa
      type: DirectoryOrCreate
  - name: myfile
    hostPath:
      path: /var/local/aaa/1.txt
      type: FileOrCreate
```

image

FEATURE STATE: Kubernetes v1.36 [stable](enabled by default)

An `image` volume source represents an OCI object (a container image or artifact) which is available on the kubelet's host machine.

An example of using the `image` volume source is:

pods/image-volumes.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: image-volume
spec:
  containers:
  - name: shell
    command: ["sleep", "infinity"]
    image: debian
    volumeMounts:
    - name: volume
      mountPath: /volume
  volumes:
  - name: volume
    image:
      reference: quay.io/crio/artifact:v2
      pullPolicy: IfNotPresent
```

The volume is resolved at Pod startup, depending on which `pullPolicy` value is provided:

`Always`

The kubelet always attempts to pull the reference. If the pull fails, the kubelet sets the Pod to `Failed`.

`Never`

The kubelet never pulls the reference and only uses a local image or artifact. The Pod becomes `Failed` if any layers of the image aren't already present locally, or if the manifest for that image isn't already cached.

`IfNotPresent`

The kubelet pulls if the reference isn't already present on disk. The Pod becomes `Failed` if the reference isn't present and the pull fails.

The volume gets re-resolved if the Pod gets deleted and recreated, which means that new remote content will become available on Pod recreation. A failure to resolve or pull the

image during Pod startup will block containers from starting and may add significant latency. Failures will be retried using normal volume backoff and will be reported on the Pod reason and message.

The types of objects that may be mounted by this volume are defined by the container runtime implementation on a host machine. At a minimum, they must include all valid types supported by the container image field. The OCI object gets mounted in a single directory (`spec.containers[*].volumeMounts[*].mountPath`) and will be mounted read-only.

Besides that:

- `subPath` or `subPathExpr` mounts for containers (`spec.containers[*].volumeMounts[*].subPath` , `spec.containers[*].volumeMounts[*].subPathExpr`) are only supported from Kubernetes v1.33.
- The field `spec.securityContext.fsGroupChangePolicy` has no effect on this volume type.
- The `AlwaysPullImages` Admission Controller does also work for this volume source like for container images.

The following fields are available for the `image` type:

`reference`

Artifact reference to be used. For example, you could specify `registry.k8s.io/conformance:v1.36.0` to load the files from the Kubernetes conformance test image. Behaves in the same way as `pod.spec.containers[*].image`. Pull secrets will be assembled in the same way as for the container image by looking up node credentials, service account image pull secrets, and Pod spec image pull secrets. This field is optional to allow higher level config management to default or override container images in workload controllers like Deployments and StatefulSets. More info about container images.

`pullPolicy`

Policy for pulling OCI objects. Possible values are: `Always`, `Never`, or `IfNotPresent`. Defaults to `Always` if `:latest` tag is specified, or `IfNotPresent` otherwise.

See the *Use an Image Volume With a Pod* example for more details on how to use the volume source.

iscsi

An `iscsi` volume allows an existing iSCSI (SCSI over IP) volume to be mounted into your Pod. Unlike `emptyDir`, which is erased when a Pod is removed, the contents of an `iscsi` volume are preserved, and the volume is merely unmounted. This means that an iscsi volume can be pre-populated with data, and that data can be shared between Pods.

> **Note:**
> You must have your own iSCSI server running with the volume created before you can use it.

A feature of iSCSI is that it can be mounted as read-only by multiple consumers simultaneously. This means that you can pre-populate a volume with your dataset and then serve it in parallel from as many Pods as you need. Unfortunately, iSCSI volumes can only be mounted by a single consumer in read-write mode. Simultaneous writers are not allowed.

local

A `local` volume represents a mounted local storage device such as a disk, partition or directory.

Local volumes can only be used as a statically created PersistentVolume. Dynamic provisioning is not supported.

Compared to `hostPath` volumes, `local` volumes are used in a durable and portable manner without manually scheduling Pods to nodes. The system is aware of the volume's node constraints by looking at the node affinity on the PersistentVolume.

However, `local` volumes are subject to the availability of the underlying node and are not suitable for all applications. If a node becomes unhealthy, then the `local` volume becomes inaccessible to the Pod. The Pod using this volume is unable to run. Applications using `local` volumes must be able to tolerate this reduced availability, as well as potential data loss, depending on the durability characteristics of the underlying disk.

The following example shows a PersistentVolume using a `local` volume and `nodeAffinity`：

```
apiVersion: v1
kind: PersistentVolume
metadata:
```

```
  name: example-pv
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-storage
  local:
    path: /mnt/disks/ssd1
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
          - example-node
```

You must set a PersistentVolume `nodeAffinity` when using `local` volumes. The Kubernetes scheduler uses the PersistentVolume `nodeAffinity` to schedule these Pods to the correct node.

PersistentVolume `volumeMode` can be set to "Block" (instead of the default value "Filesystem") to expose the local volume as a raw block device.

When using local volumes, it is recommended to create a StorageClass with `volumeBindingMode` set to `WaitForFirstConsumer`. For more details, see the local StorageClass example. Delaying volume binding ensures that the PersistentVolumeClaim binding decision will also be evaluated with any other node constraints the Pod may have, such as node resource requirements, node selectors, Pod affinity, and Pod anti-affinity.

An external static provisioner can be run separately for improved management of the local volume lifecycle. Note that this provisioner does not support dynamic provisioning yet. For an example on how to run an external local provisioner, see the local volume provisioner user guide.

> **Note:**

> The local PersistentVolume requires manual cleanup and deletion by the user if the external static provisioner is not used to manage the volume lifecycle.

nfs

An `nfs` volume allows an existing NFS (Network File System) share to be mounted into a Pod. Unlike `emptyDir`, which is erased when a Pod is removed, the contents of an `nfs` volume are preserved, and the volume is merely unmounted. This means that an NFS volume can be pre-populated with data, and that data can be shared between Pods. NFS can be mounted by multiple writers simultaneously.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /my-nfs-data
      name: test-volume
  volumes:
  - name: test-volume
    nfs:
      server: my-nfs-server.example.com
      path: /my-nfs-volume
      readOnly: true
```

> **Note:**
> You must have your own NFS server running with the share exported before you can use it.
>
> Also note that you can't specify NFS mount options in a Pod spec. You can either set mount options server-side or use /etc/nfsmount.conf. You can also mount NFS volumes via PersistentVolumes, which do allow you to set mount options.

persistentVolumeClaim

A `persistentVolumeClaim` volume is used to mount a PersistentVolume into a Pod. PersistentVolumeClaims are a way for users to "claim" durable storage (such as an iSCSI volume) without knowing the details of the particular cloud environment.

See the information about PersistentVolumes for more details.

portworxVolume (deprecated)

FEATURE STATE: Kubernetes v1.25 [deprecated]

A `portworxVolume` is an elastic block storage layer that runs hyperconverged with Kubernetes. Portworx fingerprints storage in a server, tiers based on capabilities, and aggregates capacity across multiple servers. Portworx runs in-guest in virtual machines or on bare metal Linux nodes.

A `portworxVolume` can be dynamically created through Kubernetes, or it can also be pre-provisioned and referenced inside a Pod. Here is an example Pod referencing a pre-provisioned Portworx volume:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-portworx-volume-pod
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /mnt
      name: pxvol
  volumes:
  - name: pxvol
    # This Portworx volume must already exist.
    portworxVolume:
      volumeID: "pxvol"
      fsType: "<fs-type>"
```

> **Note:**
> Make sure you have an existing PortworxVolume with the name `pxvol` before using it in the Pod.

Portworx CSI migration

> ⓘ **FEATURE STATE:** Kubernetes v1.33 [stable](enabled by default)

In Kubernetes 1.36, all operations for the in-tree Portworx volumes are redirected to the `pxd.portworx.com` Container Storage Interface (CSI) Driver by default.
Portworx CSI Driver must be installed on the cluster.

projected

A projected volume maps several existing volume sources into the same directory. For more details, see projected volumes.

secret

A `secret` volume is used to pass sensitive information, such as passwords, to Pods. You can store secrets in the Kubernetes API and mount them as files for use by Pods without coupling to Kubernetes directly. `secret` volumes are backed by tmpfs (a RAM-backed filesystem), so they are never written to non-volatile storage.

> **Note:**
> - You must create a Secret in the Kubernetes API before you can use it.
> - A Secret is always mounted as `readOnly`.
> - A container using a Secret as a `subPath` volume mount will not receive Secret updates.

For more details, see Configuring Secrets.

Using subPath

Sometimes, it is useful to share one volume for multiple uses in a single Pod. The `volumeMounts[*].subPath` property specifies a sub-path inside the referenced volume instead of its root.

The following example shows how to configure a Pod with a LAMP stack (Linux, Apache, MySQL, PHP) using a single, shared volume. This sample `subPath` configuration is not recommended for production use.

The PHP application's code and assets map to the volume's `html` folder and the MySQL database is stored in the volume's `mysql` folder. For example:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-lamp-site
spec:
    containers:
    - name: mysql
      image: mysql
      env:
      - name: MYSQL_ROOT_PASSWORD
        value: "rootpasswd"
      volumeMounts:
      - mountPath: /var/lib/mysql
        name: site-data
        subPath: mysql
    - name: php
      image: php:7.0-apache
      volumeMounts:
      - mountPath: /var/www/html
        name: site-data
        subPath: html
    volumes:
    - name: site-data
      persistentVolumeClaim:
        claimName: my-lamp-site-data
```

Using subPath with expanded environment variables

FEATURE STATE: Kubernetes v1.17 [stable]

Use the `subPathExpr` field to construct `subPath` directory names from downward API environment variables. The `subPath` and `subPathExpr` properties are mutually exclusive.

In this example, a `Pod` uses `subPathExpr` to create a directory `pod1` within the `hostPath` volume `/var/log/pods`. The `hostPath` volume takes the `Pod` name from the `downwardAPI`. The host directory `/var/log/pods/pod1` is mounted at `/logs` in the container.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod1
spec:
  containers:
  - name: container1
    env:
    - name: POD_NAME
      valueFrom:
        fieldRef:
          apiVersion: v1
          fieldPath: metadata.name
    image: busybox:1.28
    command: [ "sh", "-c", "while [ true ]; do echo 'Hello'; sleep 10; done | tee -a
    volumeMounts:
    - name: workdir1
      mountPath: /logs
      # The variable expansion uses round brackets (not curly brackets).
      subPathExpr: $(POD_NAME)
  restartPolicy: Never
  volumes:
  - name: workdir1
    hostPath:
      path: /var/log/pods
```

Resources

The storage medium (such as Disk or SSD) of an `emptyDir` volume is determined by the medium of the filesystem holding the kubelet root dir (typically `/var/lib/kubelet`). There is no limit on how much space an `emptyDir` or `hostPath` volume can consume, and no isolation between containers or Pods.

To learn about requesting space using a resource specification, see how to manage resources.

Out-of-tree volume plugins

The out-of-tree volume plugins include Container Storage Interface (CSI), and also FlexVolume (which is deprecated). These plugins enable storage vendors to create custom storage plugins without adding their plugin source code to the Kubernetes repository.

Previously, all volume plugins were "in-tree". The "in-tree" plugins were built, linked, compiled, and shipped with the core Kubernetes binaries. This meant that adding a new storage system to Kubernetes (a volume plugin) required checking code into the core Kubernetes code repository.

Both CSI and FlexVolume allow volume plugins to be developed independently of the Kubernetes code base, and deployed (installed) on Kubernetes clusters as extensions.

For storage vendors looking to create an out-of-tree volume plugin, please refer to the volume plugin FAQ.

csi

Container Storage Interface (CSI) defines a standard interface for container orchestration systems (like Kubernetes) to expose arbitrary storage systems to their container workloads.

Please read the CSI design proposal for more information.

> **Note:**
> Support for CSI spec versions 0.2 and 0.3 is deprecated in Kubernetes v1.13 and will be removed in a future release.

> **Note:**

> CSI drivers may not be compatible across all Kubernetes releases. Please check the specific CSI driver's documentation for supported deployment steps for each Kubernetes release and a compatibility matrix.

Once a CSI-compatible volume driver is deployed on a Kubernetes cluster, users may use the `csi` volume type to attach or mount the volumes exposed by the CSI driver.

A `csi` volume can be used in a Pod in three different ways:

- through a reference to a PersistentVolumeClaim
- with a generic ephemeral volume
- with a CSI ephemeral volume if the driver supports that

The following fields are available to storage administrators to configure a CSI persistent volume:

- `driver` : A string value that specifies the name of the volume driver to use. This value must correspond to the value returned in the `GetPluginInfoResponse` by the CSI driver as defined in the CSI spec. It is used by Kubernetes to identify which CSI driver to call out to, and by CSI driver components to identify which PV objects belong to the CSI driver.
- `volumeHandle` : A string value that uniquely identifies the volume. This value must correspond to the value returned in the `volume.id` field of the `CreateVolumeResponse` by the CSI driver as defined in the CSI spec. The value is passed as `volume_id` in all calls to the CSI volume driver when referencing the volume.
- `readOnly` : An optional boolean value indicating whether the volume is to be "ControllerPublished" (attached) as read-only. Default is false. This value is passed to the CSI driver via the `readonly` field in the `ControllerPublishVolumeRequest` .
- `fsType` : If the PV's `VolumeMode` is `Filesystem` , then this field may be used to specify the filesystem that should be used to mount the volume. If the volume has not been formatted and formatting is supported, this value will be used to format the volume. This value is passed to the CSI driver via the `VolumeCapability` field of `ControllerPublishVolumeRequest` , `NodeStageVolumeRequest` , and `NodePublishVolumeRequest` .
- `volumeAttributes` : A map of string to string that specifies static properties of a volume. This map must correspond to the map returned in the `volume.attributes` field of the `CreateVolumeResponse` by the CSI driver as defined in the CSI spec. The map is passed to the CSI driver via the `volume_context` field in the `ControllerPublishVolumeRequest` , `NodeStageVolumeRequest` , and `NodePublishVolumeRequest` .

- `controllerPublishSecretRef` : A reference to the secret object containing sensitive information to pass to the CSI driver to complete the CSI `ControllerPublishVolume` and `ControllerUnpublishVolume` calls. This field is optional, and may be empty if no secret is required. If the Secret contains more than one secret, all secrets are passed.
- `nodeExpandSecretRef` : A reference to the secret containing sensitive information to pass to the CSI driver to complete the CSI `NodeExpandVolume` call. This field is optional and may be empty if no secret is required. If the object contains more than one secret, all secrets are passed. When you have configured secret data for node-initiated volume expansion, the kubelet passes that data via the `NodeExpandVolume()` call to the CSI driver. All supported versions of Kubernetes offer the `nodeExpandSecretRef` field, and have it available by default. Kubernetes releases prior to v1.25 did not include this support.
- Enable the feature gate named `CSINodeExpandSecret` for each kube-apiserver and for the kubelet on every node. Since Kubernetes version 1.27, this feature has been enabled by default and no explicit enablement of the feature gate is required. You must also be using a CSI driver that supports or requires secret data during node-initiated storage resize operations.
- `nodePublishSecretRef` : A reference to the secret object containing sensitive information to pass to the CSI driver to complete the CSI `NodePublishVolume` call. This field is optional and may be empty if no secret is required. If the secret object contains more than one secret, all secrets are passed.
- `nodeStageSecretRef` : A reference to the secret object containing sensitive information to pass to the CSI driver to complete the CSI `NodeStageVolume` call. This field is optional and may be empty if no secret is required. If the Secret contains more than one secret, all secrets are passed.

CSI raw block volume support

ⓘ **FEATURE STATE:** Kubernetes v1.18 [stable]

Vendors with external CSI drivers can implement raw block volume support in Kubernetes workloads.

You can set up your PersistentVolume/PersistentVolumeClaim with raw block volume support as usual, without any CSI-specific changes.

CSI ephemeral volumes

FEATURE STATE: Kubernetes v1.25 [stable]

You can directly configure CSI volumes within the Pod specification. Volumes specified in this way are ephemeral and do not persist across Pod restarts. See Ephemeral Volumes for more information.

For more information on how to develop a CSI driver, refer to the kubernetes-csi documentation

Windows CSI proxy

FEATURE STATE: Kubernetes v1.22 [stable]

CSI node plugins need to perform various privileged operations like scanning of disk devices and mounting of file systems. These operations differ for each host operating system. For Linux worker nodes, containerized CSI node plugins are typically deployed as privileged containers. For Windows worker nodes, privileged operations for containerized CSI node plugins are supported using csi-proxy, a community-managed, stand-alone binary that needs to be pre-installed on each Windows node.

For more details, refer to the deployment guide of the CSI plugin you wish to deploy.

Migrating to CSI drivers from in-tree plugins

FEATURE STATE: Kubernetes v1.25 [stable]

The `CSIMigration` feature directs operations against existing in-tree plugins to corresponding CSI plugins (which are expected to be installed and configured). As a result, operators do not have to make any configuration changes to existing Storage Classes, PersistentVolumes, or PersistentVolumeClaims (referring to in-tree plugins) when transitioning to a CSI driver that supersedes an in-tree plugin.

> **Note:**
> Existing PVs created by an in-tree volume plugin can still be used in the future without any configuration changes, even after the migration to CSI is completed for

> that volume type, and even after you upgrade to a version of Kubernetes that doesn't have compiled-in support for that kind of storage.
>
> As part of that migration, you - or another cluster administrator - **must** have installed and configured the appropriate CSI driver for that storage. The core of Kubernetes does not install that software for you.
>
> ---
>
> After that migration, you can also define new PVCs and PVs that refer to the legacy, built-in storage integrations. Provided you have the appropriate CSI driver installed and configured, the PV creation continues to work, even for brand-new volumes. The actual storage management now happens through the CSI driver.

The operations and features that are supported include: provisioning/delete, attach/detach, mount/unmount, and resizing of volumes.

In-tree plugins that support `CSIMigration` and have a corresponding CSI driver implemented are listed in Types of Volumes.

flexVolume (deprecated)

ⓘ **FEATURE STATE:** Kubernetes v1.23 [deprecated]

FlexVolume is an out-of-tree plugin interface that uses an exec-based model to interface with storage drivers. The FlexVolume driver binaries must be installed in a pre-defined volume plugin path on each node, and in some cases, the control plane nodes as well.

Pods interact with FlexVolume drivers through the `flexVolume` in-tree volume plugin.

The following FlexVolume plugins, deployed as PowerShell scripts on the host, support Windows nodes:

- SMB
- iSCSI

> **Note:**
> FlexVolume is deprecated. Using an out-of-tree CSI driver is the recommended way to integrate external storage with Kubernetes.

> Maintainers of the FlexVolume driver should implement a CSI Driver and help migrate users of FlexVolume drivers to CSI. Users of FlexVolume should move their workloads to use the equivalent CSI Driver.

Mount propagation

> **Caution:**
>
> Mount propagation is a low-level feature that does not work consistently on all volume types. The Kubernetes project recommends only using mount propagation with `hostPath` or memory-backed `emptyDir` volumes. See Kubernetes issue #95049 for more context.

Mount propagation allows for sharing volumes mounted by a container to other containers in the same Pod, or even to other Pods on the same node.

Mount propagation of a volume is controlled by the `mountPropagation` field in `containers[*].volumeMounts`. Its values are:

- `None` - This volume mount will not receive any subsequent mounts that are mounted to this volume or any of its subdirectories by the host. In a similar fashion, no mounts created by the container will be visible on the host. This is the default mode.

  This mode is equal to `rprivate` mount propagation as described in `mount(8)`

  However, the CRI runtime may choose `rslave` mount propagation (i.e., `HostToContainer`) when `rprivate` propagation is not applicable. cri-dockerd (Docker) is known to choose `rslave` mount propagation when the mount source contains the Docker daemon's root directory (`/var/lib/docker`).

- `HostToContainer` - This volume mount will receive all subsequent mounts that are mounted to this volume or any of its subdirectories.

  In other words, if the host mounts anything inside the volume mount, the container will see it mounted there.

  Similarly, if any Pod with `Bidirectional` mount propagation to the same volume mounts anything there, the container with `HostToContainer` mount propagation will see it.

This mode is equal to `rslave` mount propagation as described in the `mount(8)`

- `Bidirectional` - This volume mount behaves the same as the `HostToContainer` mount. In addition, all volume mounts created by the container will be propagated back to the host and to all containers of all Pods that use the same volume.

  A typical use case for this mode is a Pod with a FlexVolume or CSI driver, or a Pod that needs to mount something on the host using a `hostPath` volume.

  This mode is equal to `rshared` mount propagation as described in the `mount(8)`

> **Warning:**
>
> `Bidirectional` mount propagation can be dangerous. It can damage the host operating system, and therefore, it is allowed only in privileged containers. Familiarity with Linux kernel behavior is strongly recommended. In addition, any volume mounts created by containers in Pods must be destroyed (unmounted) by the containers on termination.

Read-only mounts

A mount can be made read-only by setting the `.spec.containers[*].volumeMounts[*].readOnly` field to `true`. This does not make the volume itself read-only, but that specific container will not be able to write to it. Other containers in the Pod may mount the same volume as read-write.

On Linux, read-only mounts are not recursively read-only by default. For example, consider a Pod that mounts the hosts `/mnt` as a `hostPath` volume. If there is another filesystem mounted read-write on `/mnt/<SUBMOUNT>` (such as tmpfs, NFS, or USB storage), the volume mounted into the container(s) will also have a writeable `/mnt/<SUBMOUNT>`, even if the mount itself was specified as read-only.

Recursive read-only mounts

ⓘ **FEATURE STATE:** Kubernetes v1.33 [stable](enabled by default)

Recursive read-only mounts can be enabled by setting the `RecursiveReadOnlyMounts` feature gate for kubelet and kube-apiserver, and setting the `.spec.containers[*].volumeMounts[*].recursiveReadOnly` field for a Pod.

The allowed values are:

- `Disabled` (default): no effect.
- `Enabled` : makes the mount recursively read-only. Needs all the following requirements to be satisfied:
  - `readOnly` is set to `true`
  - `mountPropagation` is unset, or set to `None`
  - The host is running with Linux kernel v5.12 or later
  - The CRI-level container runtime supports recursive read-only mounts
  - The OCI-level container runtime supports recursive read-only mounts.

  It will fail if any of these is not true.
- `IfPossible` : attempts to apply `Enabled` , and falls back to `Disabled` if the feature is not supported by the kernel or the runtime class.

Example:

storage/rro.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: rro
spec:
  volumes:
    - name: mnt
      hostPath:
        # tmpfs is mounted on /mnt/tmpfs
        path: /mnt
  containers:
    - name: busybox
      image: busybox
      args: ["sleep", "infinity"]
      volumeMounts:
        # /mnt-rro/tmpfs is not writable
        - name: mnt
```

```
      mountPath: /mnt-rro
      readOnly: true
      mountPropagation: None
      recursiveReadOnly: Enabled
    # /mnt-ro/tmpfs is writable
    - name: mnt
      mountPath: /mnt-ro
      readOnly: true
    # /mnt-rw/tmpfs is writable
    - name: mnt
      mountPath: /mnt-rw
```

When this property is recognized by kubelet and kube-apiserver, the `.status.containerStatuses[*].volumeMounts[*].recursiveReadOnly` field is set to either `Enabled` or `Disabled`.

Implementations

> **Note:** This section links to third party projects that provide functionality required by Kubernetes. The Kubernetes project authors aren't responsible for these projects, which are listed alphabetically. To add a project to this list, read the content guide before submitting a change. More information.

The following container runtimes are known to support recursive read-only mounts.

CRI-level:

- containerd, since v2.0
- CRI-O, since v1.30

OCI-level:

- runc, since v1.1
- crun, since v1.8.6

What's next

Follow an example of deploying WordPress and MySQL with Persistent Volumes.

2 - Persistent Volumes

This document describes *persistent volumes* in Kubernetes. Familiarity with volumes, StorageClasses and VolumeAttributesClasses is suggested.

Introduction

Managing storage is a distinct problem from managing compute instances. The PersistentVolume subsystem provides an API for users and administrators that abstracts details of how storage is provided from how it is consumed. To do this, we introduce two new API resources: PersistentVolume and PersistentVolumeClaim.

A *PersistentVolume* (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes. It is a resource in the cluster just like a node is a cluster resource. PVs are volume plugins like Volumes, but have a lifecycle independent of any individual Pod that uses the PV. This API object captures the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system.

A *PersistentVolumeClaim* (PVC) is a request for storage by a user. It is similar to a Pod. Pods consume node resources and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory). Claims can request specific size and access modes (e.g., they can be mounted ReadWriteOnce, ReadOnlyMany, ReadWriteMany, or ReadWriteOncePod, see AccessModes).

While PersistentVolumeClaims allow a user to consume abstract storage resources, it is common that users need PersistentVolumes with varying properties, such as performance, for different problems. Cluster administrators need to be able to offer a variety of PersistentVolumes that differ in more ways than size and access modes, without exposing users to the details of how those volumes are implemented. For these needs, there is the *StorageClass* resource.

See the detailed walkthrough with working examples.

Lifecycle of a volume and claim

PVs are resources in the cluster. PVCs are requests for those resources and also act as claim checks to the resource. The interaction between PVs and PVCs follows this lifecycle:

Provisioning

There are two ways PVs may be provisioned: statically or dynamically.

Static

A cluster administrator creates a number of PVs. They carry the details of the real storage, which is available for use by cluster users. They exist in the Kubernetes API and are available for consumption.

Dynamic

When none of the static PVs the administrator created match a user's PersistentVolumeClaim, the cluster may try to dynamically provision a volume specially for the PVC. This provisioning is based on StorageClasses: the PVC must request a storage class and the administrator must have created and configured that class for dynamic provisioning to occur. Claims that request the class `""` effectively disable dynamic provisioning for themselves.

To enable dynamic storage provisioning based on storage class, the cluster administrator needs to enable the `DefaultStorageClass` admission controller on the API server. This can be done, for example, by ensuring that `DefaultStorageClass` is among the comma-delimited, ordered list of values for the `--enable-admission-plugins` flag of the API server component. For more information on API server command-line flags, check kube-apiserver documentation.

Binding

A user creates, or in the case of dynamic provisioning, has already created, a PersistentVolumeClaim with a specific amount of storage requested and with certain access modes. A control loop in the control plane watches for new PVCs, finds a matching PV (if possible), and binds them together. If a PV was dynamically provisioned for a new PVC, the loop will always bind that PV to the PVC. Otherwise, the user will always get at least what they asked for, but the volume may be in excess of what was requested. Once bound, PersistentVolumeClaim binds are exclusive, regardless of how they were bound. A PVC to PV binding is a one-to-one mapping, using a ClaimRef which is a bi-directional binding between the PersistentVolume and the PersistentVolumeClaim.

Claims will remain unbound indefinitely if a matching volume does not exist. Claims will be bound as matching volumes become available. For example, a cluster provisioned with

many 50Gi PVs would not match a PVC requesting 100Gi. The PVC can be bound when a 100Gi PV is added to the cluster.

Using

Pods use claims as volumes. The cluster inspects the claim to find the bound volume and mounts that volume for a Pod. For volumes that support multiple access modes, the user specifies which mode is desired when using their claim as a volume in a Pod.

Once a user has a claim and that claim is bound, the bound PV belongs to the user for as long as they need it. Users schedule Pods and access their claimed PVs by including a `persistentVolumeClaim` section in a Pod's `volumes` block. See Claims As Volumes for more details on this.

Storage Object in Use Protection

The purpose of the Storage Object in Use Protection feature is to ensure that PersistentVolumeClaims (PVCs) in active use by a Pod and PersistentVolume (PVs) that are bound to PVCs are not removed from the system, as this may result in data loss.

> **Note:**
> PVC is in active use by a Pod when a Pod object exists that is using the PVC.

If a user deletes a PVC in active use by a Pod, the PVC is not removed immediately. PVC removal is postponed until the PVC is no longer actively used by any Pods. Also, if an admin deletes a PV that is bound to a PVC, the PV is not removed immediately. PV removal is postponed until the PV is no longer bound to a PVC.

You can see that a PVC is protected when the PVC's status is `Terminating` and the `Finalizers` list includes `kubernetes.io/pvc-protection`:

```
kubectl describe pvc hostpath
Name:          hostpath
Namespace:     default
StorageClass:  example-hostpath
Status:        Terminating
Volume:
Labels:        <none>
Annotations:   volume.beta.kubernetes.io/storage-class=example-hostpath
```

```
                volume.beta.kubernetes.io/storage-provisioner=example.com/hostpath
Finalizers:     [kubernetes.io/pvc-protection]
...
```

You can see that a PV is protected when the PV's status is `Terminating` and the `Finalizers` list includes `kubernetes.io/pv-protection` too:

```
kubectl describe pv task-pv-volume
Name:            task-pv-volume
Labels:          type=local
Annotations:     <none>
Finalizers:      [kubernetes.io/pv-protection]
StorageClass:    standard
Status:          Terminating
Claim:
Reclaim Policy:  Delete
Access Modes:    RWO
Capacity:        1Gi
Message:
Source:
    Type:          HostPath (bare host directory volume)
    Path:          /tmp/data
    HostPathType:
Events:            <none>
```

Reclaiming

When a user is done with their volume, they can delete the PVC objects from the API that allows reclamation of the resource. The reclaim policy for a PersistentVolume tells the cluster what to do with the volume after it has been released of its claim. Currently, volumes can either be Retained, Recycled, or Deleted.

Retain

The `Retain` reclaim policy allows for manual reclamation of the resource. When the PersistentVolumeClaim is deleted, the PersistentVolume still exists and the volume is considered "released". But it is not yet available for another claim because the previous claimant's data remains on the volume. An administrator can manually reclaim the volume with the following steps.

1. Delete the PersistentVolume. The associated storage asset in external infrastructure still exists after the PV is deleted.
2. Manually clean up the data on the associated storage asset accordingly.
3. Manually delete the associated storage asset.

If you want to reuse the same storage asset, create a new PersistentVolume with the same storage asset definition.

Delete

For volume plugins that support the `Delete` reclaim policy, deletion removes both the PersistentVolume object from Kubernetes, as well as the associated storage asset in the external infrastructure. Volumes that were dynamically provisioned inherit the reclaim policy of their StorageClass, which defaults to `Delete`. The administrator should configure the StorageClass according to users' expectations; otherwise, the PV must be edited or patched after it is created. See Change the Reclaim Policy of a PersistentVolume.

Recycle

> **Warning:**
>
> The `Recycle` reclaim policy is deprecated. Instead, the recommended approach is to use dynamic provisioning.

If supported by the underlying volume plugin, the `Recycle` reclaim policy performs a basic scrub (`rm -rf /thevolume/*`) on the volume and makes it available again for a new claim.

However, an administrator can configure a custom recycler Pod template using the Kubernetes controller manager command line arguments as described in the reference. The custom recycler Pod template must contain a `volumes` specification, as shown in the example below:

```
apiVersion: v1
kind: Pod
metadata:
  name: pv-recycler
  namespace: default
spec:
```

```
  restartPolicy: Never
  volumes:
  - name: vol
    hostPath:
      path: /any/path/it/will/be/replaced
  containers:
  - name: pv-recycler
    image: "registry.k8s.io/busybox"
    command: ["/bin/sh", "-c", "test -e /scrub && rm -rf /scrub/..?* /scrub/.[!.]* /s
    volumeMounts:
    - name: vol
      mountPath: /scrub
```

However, the particular path specified in the custom recycler Pod template in the `volumes` part is replaced with the particular path of the volume that is being recycled.

PersistentVolume deletion protection finalizer

FEATURE STATE: Kubernetes v1.33 [stable](enabled by default)

Finalizers can be added on a PersistentVolume to ensure that PersistentVolumes having `Delete` reclaim policy are deleted only after the backing storage are deleted.

The finalizer `external-provisioner.volume.kubernetes.io/finalizer` (introduced in v1.31) is added to both dynamically provisioned and statically provisioned CSI volumes.

The finalizer `kubernetes.io/pv-controller` (introduced in v1.31) is added to dynamically provisioned in-tree plugin volumes and skipped for statically provisioned in-tree plugin volumes.

The following is an example of dynamically provisioned in-tree plugin volume:

```
kubectl describe pv pvc-74a498d6-3929-47e8-8c02-078c1ece4d78
Name:            pvc-74a498d6-3929-47e8-8c02-078c1ece4d78
Labels:          <none>
Annotations:     kubernetes.io/createdby: vsphere-volume-dynamic-provisioner
                 pv.kubernetes.io/bound-by-controller: yes
```

```
                pv.kubernetes.io/provisioned-by: kubernetes.io/vsphere-volume
Finalizers:     [kubernetes.io/pv-protection kubernetes.io/pv-controller]
StorageClass:   vcp-sc
Status:         Bound
Claim:          default/vcp-pvc-1
Reclaim Policy: Delete
Access Modes:   RWO
VolumeMode:     Filesystem
Capacity:       1Gi
Node Affinity:  <none>
Message:
Source:
    Type:               vSphereVolume (a Persistent Disk resource in vSphere)
    VolumePath:         [vsanDatastore] d49c4a62-166f-ce12-c464-020077ba5d46/kubernet
    FSType:             ext4
    StoragePolicyName:  vSAN Default Storage Policy
Events:                 <none>
```

The finalizer `external-provisioner.volume.kubernetes.io/finalizer` is added for CSI volumes. The following is an example:

```
Name:            pvc-2f0bab97-85a8-4552-8044-eb8be45cf48d
Labels:          <none>
Annotations:     pv.kubernetes.io/provisioned-by: csi.vsphere.vmware.com
Finalizers:      [kubernetes.io/pv-protection external-provisioner.volume.kubernetes.
StorageClass:    fast
Status:          Bound
Claim:           demo-app/nginx-logs
Reclaim Policy:  Delete
Access Modes:    RWO
VolumeMode:      Filesystem
Capacity:        200Mi
Node Affinity:   <none>
Message:
Source:
    Type:              CSI (a Container Storage Interface (CSI) volume source)
    Driver:            csi.vsphere.vmware.com
    FSType:            ext4
    VolumeHandle:      44830fa8-79b4-406b-8b58-621ba25353fd
    ReadOnly:          false
    VolumeAttributes:      storage.kubernetes.io/csiProvisionerIdentity=1648442357185
                           type=vSphere CNS Block Volume
```

```
Events:                <none>
```

When the `CSIMigration{provider}` feature flag is enabled for a specific in-tree volume plugin, the `kubernetes.io/pv-controller` finalizer is replaced by the `external-provisioner.volume.kubernetes.io/finalizer` finalizer.

The finalizers ensure that the PV object is removed only after the volume is deleted from the storage backend provided the reclaim policy of the PV is `Delete`. This also ensures that the volume is deleted from storage backend irrespective of the order of deletion of PV and PVC.

Reserving a PersistentVolume

The control plane can bind PersistentVolumeClaims to matching PersistentVolumes in the cluster. However, if you want a PVC to bind to a specific PV, you need to pre-bind them.

By specifying a PersistentVolume in a PersistentVolumeClaim, you declare a binding between that specific PV and PVC. If the PersistentVolume exists and has not reserved PersistentVolumeClaims through its `claimRef` field, then the PersistentVolume and PersistentVolumeClaim will be bound.

The binding happens regardless of some volume matching criteria, including node affinity. The control plane still checks that storage class, access modes, and requested storage size are valid.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: foo-pvc
  namespace: foo
spec:
  storageClassName: "" # Empty string must be explicitly set otherwise default Storag
  volumeName: foo-pv
  ...
```

This method does not guarantee any binding privileges to the PersistentVolume. If other PersistentVolumeClaims could use the PV that you specify, you first need to reserve that

storage volume. Specify the relevant PersistentVolumeClaim in the `claimRef` field of the PV so that other PVCs can not bind to it.

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: foo-pv
spec:
  storageClassName: ""
  claimRef:
    name: foo-pvc
    namespace: foo
  ...
```

This is useful if you want to consume PersistentVolumes that have their `persistentVolumeReclaimPolicy` set to `Retain`, including cases where you are reusing an existing PV.

Expanding Persistent Volumes Claims

FEATURE STATE: Kubernetes v1.24 [stable]

Support for expanding PersistentVolumeClaims (PVCs) is enabled by default. You can expand the following types of volumes:

- csi (including some CSI migrated volume types)
- flexVolume (deprecated)
- portworxVolume (deprecated)

You can only expand a PVC if its storage class's `allowVolumeExpansion` field is set to true.

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: example-vol-default
provisioner: vendor-name.example/magicstorage
parameters:
```

```
  resturl: "http://192.168.10.100:8080"
  restuser: ""
  secretNamespace: ""
  secretName: ""
allowVolumeExpansion: true
```

To request a larger volume for a PVC, edit the PVC object and specify a larger size. This triggers expansion of the volume that backs the underlying PersistentVolume. A new PersistentVolume is never created to satisfy the claim. Instead, an existing volume is resized.

> **Warning:**
>
> Directly editing the size of a PersistentVolume can prevent an automatic resize of that volume. If you edit the capacity of a PersistentVolume, and then edit the `.spec` of a matching PersistentVolumeClaim to make the size of the PersistentVolumeClaim match the PersistentVolume, then no storage resize happens. The Kubernetes control plane will see that the desired state of both resources matches, conclude that the backing volume size has been manually increased and that no resize is necessary.

CSI Volume expansion

ⓘ **FEATURE STATE:** Kubernetes v1.24 [stable]

Support for expanding CSI volumes is enabled by default but it also requires a specific CSI driver to support volume expansion. Refer to documentation of the specific CSI driver for more information.

Resizing a volume containing a file system

You can only resize volumes containing a file system if the file system is XFS, Ext3, or Ext4.

When a volume contains a file system, the file system is only resized when a new Pod is using the PersistentVolumeClaim in `ReadWrite` mode. File system expansion is either done when a Pod is starting up or when a Pod is running and the underlying file system supports online expansion.

FlexVolumes (deprecated since Kubernetes v1.23) allow resize if the driver is configured with the `RequiresFSResize` capability to `true`. The FlexVolume can be resized on Pod restart.

Resizing an in-use PersistentVolumeClaim

FEATURE STATE: Kubernetes v1.24 [stable]

In this case, you don't need to delete and recreate a Pod or deployment that is using an existing PVC. Any in-use PVC automatically becomes available to its Pod as soon as its file system has been expanded. This feature has no effect on PVCs that are not in use by a Pod or deployment. You must create a Pod that uses the PVC before the expansion can complete.

Similar to other volume types - FlexVolume volumes can also be expanded when in-use by a Pod.

> **Note:**
> FlexVolume resize is possible only when the underlying driver supports resize.

Recovering from Failure when Expanding Volumes

If a user specifies a new size that is too big to be satisfied by underlying storage system, expansion of PVC will be continuously retried until user or cluster administrator takes some action. This can be undesirable and hence Kubernetes provides following methods of recovering from such failures.

Manually with Cluster Administrator access

By requesting expansion to smaller size

If expanding underlying storage fails, the cluster administrator can manually recover the Persistent Volume Claim (PVC) state and cancel the resize requests. Otherwise, the resize requests are continuously retried by the controller without administrator intervention.

1. Mark the PersistentVolume(PV) that is bound to the PersistentVolumeClaim(PVC) with `Retain` reclaim policy.

2. Delete the PVC. Since PV has `Retain` reclaim policy - we will not lose any data when we recreate the PVC.
3. Delete the `claimRef` entry from PV specs, so as new PVC can bind to it. This should make the PV `Available`.
4. Re-create the PVC with smaller size than PV and set `volumeName` field of the PVC to the name of the PV. This should bind new PVC to existing PV.
5. Don't forget to restore the reclaim policy of the PV.

Types of Persistent Volumes

PersistentVolume types are implemented as plugins. Kubernetes currently supports the following plugins:

- `csi` - Container Storage Interface (CSI)
- `fc` - Fibre Channel (FC) storage
- `hostPath` - HostPath volume (for single node testing only; WILL NOT WORK in a multi-node cluster; consider using `local` volume instead)
- `iscsi` - iSCSI (SCSI over IP) storage
- `local` - local storage devices mounted on nodes.
- `nfs` - Network File System (NFS) storage

The following types of PersistentVolume are deprecated but still available. If you are using these volume types except for `flexVolume`, `cephfs` and `rbd`, please install corresponding CSI drivers.

- `awsElasticBlockStore` - AWS Elastic Block Store (EBS) (**migration on by default** starting v1.23)
- `azureDisk` - Azure Disk (**migration on by default** starting v1.23)
- `azureFile` - Azure File (**migration on by default** starting v1.24)
- `cinder` - Cinder (OpenStack block storage) (**migration on by default** starting v1.21)
- `flexVolume` - FlexVolume (**deprecated** starting v1.23, no migration plan and no plan to remove support)
- `gcePersistentDisk` - GCE Persistent Disk (**migration on by default** starting v1.23)
- `portworxVolume` - Portworx volume (**migration on by default** starting v1.31)
- `vsphereVolume` - vSphere VMDK volume (**migration on by default** starting v1.25)

Older versions of Kubernetes also supported the following in-tree PersistentVolume types:

- `cephfs` (**not available** starting v1.31)
- `flocker` - Flocker storage. (**not available** starting v1.25)
- `glusterfs` - GlusterFS storage. (**not available** starting v1.26)
- `photonPersistentDisk` - Photon controller persistent disk. (**not available** starting v1.15)
- `quobyte` - Quobyte volume. (**not available** starting v1.25)
- `rbd` - Rados Block Device (RBD) volume (**not available** starting v1.31)
- `scaleIO` - ScaleIO volume. (**not available** starting v1.21)
- `storageos` - StorageOS volume. (**not available** starting v1.25)

Persistent Volumes

Each PV contains a spec and status, which is the specification and status of the volume. The name of a PersistentVolume object must be a valid DNS subdomain name.

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0003
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /tmp
    server: 172.17.0.2
```

> **Note:**
> Helper programs relating to the volume type may be required for consumption of a PersistentVolume within a cluster. In this example, the PersistentVolume is of type

> NFS and the helper program /sbin/mount.nfs is required to support the mounting of NFS filesystems.

Capacity

Generally, a PV will have a specific storage capacity. This is set using the PV's `capacity` attribute which is a Quantity value.

Currently, storage size is the only resource that can be set or requested. Future attributes may include IOPS, throughput, etc.

Volume Mode

FEATURE STATE: Kubernetes v1.18 [stable]

Kubernetes supports two `volumeModes` of PersistentVolumes: `Filesystem` and `Block`.

`volumeMode` is an optional API parameter. `Filesystem` is the default mode used when `volumeMode` parameter is omitted.

A volume with `volumeMode: Filesystem` is *mounted* into Pods into a directory. If the volume is backed by a block device and the device is empty, Kubernetes creates a filesystem on the device before mounting it for the first time.

You can set the value of `volumeMode` to `Block` to use a volume as a raw block device. Such volume is presented into a Pod as a block device, without any filesystem on it. This mode is useful to provide a Pod the fastest possible way to access a volume, without any filesystem layer between the Pod and the volume. On the other hand, the application running in the Pod must know how to handle a raw block device. See Raw Block Volume Support for an example on how to use a volume with `volumeMode: Block` in a Pod.

Access Modes

A PersistentVolume can be mounted on a host in any way supported by the resource provider. As shown in the table below, providers will have different capabilities and each PV's access modes are set to the specific modes supported by that particular volume. For example, NFS can support multiple read/write clients, but a specific NFS PV might be exported on the server as read-only. Each PV gets its own set of access modes describing that specific PV's capabilities.

The access modes are:

`ReadWriteOnce`

the volume can be mounted as read-write by a single node. ReadWriteOnce access mode still can allow multiple pods to access (read from or write to) that volume when the pods are running on the same node. For single pod access, please see ReadWriteOncePod.

`ReadOnlyMany`

the volume can be mounted as read-only by many nodes.

`ReadWriteMany`

the volume can be mounted as read-write by many nodes.

`ReadWriteOncePod`

ⓘ **FEATURE STATE:** Kubernetes v1.29 [stable]

the volume can be mounted as read-write by a single Pod. Use ReadWriteOncePod access mode if you want to ensure that only one pod across the whole cluster can read that PVC or write to it.

> **Note:**
> The `ReadWriteOncePod` access mode is only supported for CSI volumes and Kubernetes version 1.22+. To use this feature you will need to update the following CSI sidecars to these versions or greater:
>
> - csi-provisioner:v3.0.0+
> - csi-attacher:v3.3.0+
> - csi-resizer:v1.3.0+

In the CLI, the access modes are abbreviated to:

- RWO - ReadWriteOnce
- ROX - ReadOnlyMany
- RWX - ReadWriteMany

- RWOP - ReadWriteOncePod

> **Note:**
> Kubernetes uses volume access modes to match PersistentVolumeClaims and PersistentVolumes. In some cases, the volume access modes also constrain where the PersistentVolume can be mounted. Volume access modes do **not** enforce write protection once the storage has been mounted. Even if the access modes are specified as ReadWriteOnce, ReadOnlyMany, or ReadWriteMany, they don't set any constraints on the volume. For example, even if a PersistentVolume is created as ReadOnlyMany, it is no guarantee that it will be read-only. If the access modes are specified as ReadWriteOncePod, the volume is constrained and can be mounted on only a single Pod.

> **Important!** A volume can only be mounted using one access mode at a time, even if it supports many.

Volume Plugin	ReadWriteOnce	ReadOnlyMany	ReadWriteMany	ReadWriteOr
AzureFile	✓	✓	✓	-
CephFS	✓	✓	✓	-
CSI	depends on the driver	depends on the driver	depends on the driver	depends on th driver
FC	✓	✓	-	-
FlexVolume	✓	✓	depends on the driver	-
HostPath	✓	-	-	-
iSCSI	✓	✓	-	-
NFS	✓	✓	✓	-
RBD	✓	✓	-	-
VsphereVolume	✓	-	- (works when Pods are collocated)	-

Volume Plugin	ReadWriteOnce	ReadOnlyMany	ReadWriteMany	ReadWriteOr
PortworxVolume	✓	-	✓	-

Class

A PV can have a class, which is specified by setting the `storageClassName` attribute to the name of a StorageClass. A PV of a particular class can only be bound to PVCs requesting that class. A PV with no `storageClassName` has no class and can only be bound to PVCs that request no particular class.

In the past, the annotation `volume.beta.kubernetes.io/storage-class` was used instead of the `storageClassName` attribute. This annotation is still working; however, it will become fully deprecated in a future Kubernetes release.

Reclaim Policy

Current reclaim policies are:

- Retain -- manual reclamation
- Recycle -- basic scrub (`rm -rf /thevolume/*`)
- Delete -- delete the volume

For Kubernetes 1.36, only `nfs` and `hostPath` volume types support recycling.

Mount Options

A Kubernetes administrator can specify additional mount options for when a Persistent Volume is mounted on a node.

> **Note:**
> Not all Persistent Volume types support mount options.

The following volume types support mount options:

- `csi` (including CSI migrated volume types)
- `iscsi`

- `nfs`

Mount options are not validated. If a mount option is invalid, the mount fails.

In the past, the annotation `volume.beta.kubernetes.io/mount-options` was used instead of the `mountOptions` attribute. This annotation is still working; however, it will become fully deprecated in a future Kubernetes release.

Node Affinity

> **Note:**
> For most volume types, you do not need to set this field. You need to explicitly set this for local volumes.

A PV can specify node affinity to define constraints that limit what nodes this volume can be accessed from. Pods that use a PV will only be scheduled to nodes that are selected by the node affinity. To specify node affinity, set `nodeAffinity` in the `.spec` of a PV. The PersistentVolume API reference has more details on this field.

Updates to node affinity

> ⓘ **FEATURE STATE:** Kubernetes v1.35 [alpha](disabled by default)

If the `MutablePVNodeAffinity` feature gate is enabled in your cluster, the `.spec.nodeAffinity` field of a PersistentVolume is mutable. This allows cluster administrators or external storage controller to update the node affinity of a PersistentVolume when the data is migrated, without interrupting the running pods.

When updating the node affinity, you should ensure that the new node affinity still matches the nodes where the volume is currently in use. For the pods violating the new affinity, if the pod is already running, it may continue to run. But Kubernetes does not support this configuration. You should terminate the violating pods soon. Due to in memory caching, the pods created after the update may still be scheduled according to the old node affinity for a short period of time.

To use this feature, you should enable the `MutablePVNodeAffinity` feature gate on the following components:

- `kube-apiserver`
- `kubelet`

Phase

A PersistentVolume will be in one of the following phases:

`Available`

a free resource that is not yet bound to a claim

`Bound`

the volume is bound to a claim

`Released`

the claim has been deleted, but the associated storage resource is not yet reclaimed by the cluster

`Failed`

the volume has failed its (automated) reclamation

You can see the name of the PVC bound to the PV using `kubectl describe persistentvolume <name>`.

Phase transition timestamp

ⓘ **FEATURE STATE:** Kubernetes v1.31 [stable](enabled by default)

The `.status` field for a PersistentVolume can include an alpha `lastPhaseTransitionTime` field. This field records the timestamp of when the volume last transitioned its phase. For newly created volumes the phase is set to `Pending` and `lastPhaseTransitionTime` is set to the current time.

PersistentVolumeClaims

Each PVC contains a spec and status, which is the specification and status of the claim. The name of a PersistentVolumeClaim object must be a valid DNS subdomain name.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
  selector:
    matchLabels:
      release: "stable"
    matchExpressions:
      - {key: environment, operator: In, values: [dev]}
```

Access Modes

Claims use the same conventions as volumes when requesting storage with specific access modes.

Volume Modes

Claims use the same convention as volumes to indicate the consumption of the volume as either a filesystem or block device.

Volume Name

Claims can use the `volumeName` field to explicitly bind to a specific PersistentVolume. You can also leave `volumeName` unset, indicating that you'd like Kubernetes to set up a new PersistentVolume that matches the claim. If the specified PV is already bound to another PVC, the binding will be stuck in a pending state.

Resources

Claims, like Pods, can request specific quantities of a resource. In this case, the request is for storage. The same resource model applies to both volumes and claims.

> **Note:**
> For `Filesystem` volumes, the storage request refers to the "outer" volume size (i.e. the allocated size from the storage backend). This means that the writeable size may be slightly lower for providers that build a filesystem on top of a block device, due to filesystem overhead. This is especially visible with XFS, where many metadata features are enabled by default.

Selector

Claims can specify a label selector to further filter the set of volumes. Only the volumes whose labels match the selector can be bound to the claim. The selector can consist of two fields:

- `matchLabels` - the volume must have a label with this value
- `matchExpressions` - a list of requirements made by specifying key, list of values, and operator that relates the key and values. Valid operators include `In`, `NotIn`, `Exists`, and `DoesNotExist`.

All of the requirements, from both `matchLabels` and `matchExpressions`, are ANDed together – they must all be satisfied in order to match.

Class

A claim can request a particular class by specifying the name of a StorageClass using the attribute `storageClassName`. Only PVs of the requested class, ones with the same `storageClassName` as the PVC, can be bound to the PVC.

PVCs don't necessarily have to request a class. A PVC with its `storageClassName` set equal to `""` is always interpreted to be requesting a PV with no class, so it can only be bound to PVs with no class (no annotation or one set equal to `""`). A PVC with no `storageClassName` is not quite the same and is treated differently by the cluster, depending on whether the `DefaultStorageClass` admission plugin is turned on.

- If the admission plugin is turned on, the administrator may specify a default StorageClass. All PVCs that have no `storageClassName` can be bound only to PVs of that default. Specifying a default StorageClass is done by setting the annotation `storageclass.kubernetes.io/is-default-class` equal to `true` in a StorageClass object.

If the administrator does not specify a default, the cluster responds to PVC creation as if the admission plugin were turned off. If more than one default StorageClass is specified, the newest default is used when the PVC is dynamically provisioned.

- If the admission plugin is turned off, there is no notion of a default StorageClass. All PVCs that have `storageClassName` set to `""` can be bound only to PVs that have `storageClassName` also set to `""`. However, PVCs with missing `storageClassName` can be updated later once default StorageClass becomes available. If the PVC gets updated it will no longer bind to PVs that have `storageClassName` also set to `""`.

See retroactive default StorageClass assignment for more details.

Depending on installation method, a default StorageClass may be deployed to a Kubernetes cluster by addon manager during installation.

When a PVC specifies a `selector` in addition to requesting a StorageClass, the requirements are ANDed together: only a PV of the requested class and with the requested labels may be bound to the PVC.

> **Note:**
> Currently, a PVC with a non-empty `selector` can't have a PV dynamically provisioned for it.

In the past, the annotation `volume.beta.kubernetes.io/storage-class` was used instead of `storageClassName` attribute. This annotation is still working; however, it won't be supported in a future Kubernetes release.

Retroactive default StorageClass assignment

FEATURE STATE: Kubernetes v1.28 [stable]

You can create a PersistentVolumeClaim without specifying a `storageClassName` for the new PVC, and you can do so even when no default StorageClass exists in your cluster. In this case, the new PVC creates as you defined it, and the `storageClassName` of that PVC remains unset until default becomes available.

When a default StorageClass becomes available, the control plane identifies any existing PVCs without `storageClassName`. For the PVCs that either have an empty value for `storageClassName` or do not have this key, the control plane then updates those PVCs to

set `storageClassName` to match the new default StorageClass. If you have an existing PVC where the `storageClassName` is `""`, and you configure a default StorageClass, then this PVC will not get updated.

In order to keep binding to PVs with `storageClassName` set to `""` (while a default StorageClass is present), you need to set the `storageClassName` of the associated PVC to `""`.

This behavior helps administrators change default StorageClass by removing the old one first and then creating or setting another one. This brief window while there is no default causes PVCs without `storageClassName` created at that time to not have any default, but due to the retroactive default StorageClass assignment this way of changing defaults is safe.

Unused PVC tracking

ⓘ **FEATURE STATE:** Kubernetes v1.36 [alpha](disabled by default)

When enabled, the PVC protection controller adds an `Unused` condition to each PersistentVolumeClaim to indicate whether it is currently referenced by any non-terminal Pod.

The condition has two states:

`Unused` with status `"True"` (reason `NoPodsUsingPVC`)

No non-terminal Pod references this PVC. The `lastTransitionTime` records when the PVC became unused.

`Unused` with status `"False"` (reason `PodUsingPVC`)

At least one non-terminal Pod currently references this PVC. The `lastTransitionTime` records when the PVC started being used.

A Pod is considered non-terminal if its phase is not `Succeeded` or `Failed`. This means that a Pending Pod (even one that has not yet been scheduled) counts as using the PVC.

The `lastTransitionTime` of the `Unused` condition can be used by cluster administrators, monitoring tools, and external controllers to identify PVCs that have been unused for a long time. For example, to find all PVCs that have been unused for more than 30 days, you

could query for PVCs where the `Unused` condition has `status: "True"` and `lastTransitionTime` is older than 30 days.

> **Note:**
> The unused duration indicated by this condition may be shorter than the actual unused time because of processing delays in the controller or because the feature was enabled after the PVC was already unused. The condition is not updated when a PVC has `deletionTimestamp` set (that is, PVCs that are being deleted).

Claims As Volumes

Pods access storage by using the claim as a volume. Claims must exist in the same namespace as the Pod using the claim. The cluster finds the claim in the Pod's namespace and uses it to get the PersistentVolume backing the claim. The volume is then mounted to the host and into the Pod.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
      - mountPath: "/var/www/html"
        name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: myclaim
```

A Note on Namespaces

PersistentVolumes binds are exclusive, and since PersistentVolumeClaims are namespaced objects, mounting claims with "Many" modes (`ROX` , `RWX`) is only possible within one namespace.

PersistentVolumes typed `hostPath`

A `hostPath` PersistentVolume uses a file or directory on the Node to emulate network-attached storage. See an example of `hostPath` typed volume.

Raw Block Volume Support

FEATURE STATE: Kubernetes v1.18 [stable]

The following volume plugins support raw block volumes, including dynamic provisioning where applicable:

- CSI (including some CSI migrated volume types)
- FC (Fibre Channel)
- iSCSI
- Local volume

PersistentVolume using a Raw Block Volume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: block-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  volumeMode: Block
  persistentVolumeReclaimPolicy: Retain
  fc:
    targetWWNs: ["50060e801049cfd1"]
    lun: 0
    readOnly: false
```

PersistentVolumeClaim requesting a Raw Block Volume

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: block-pvc
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Block
  resources:
    requests:
      storage: 10Gi
```

Pod specification adding Raw Block Device path in container

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-block-volume
spec:
  containers:
    - name: fc-container
      image: fedora:26
      command: ["/bin/sh", "-c"]
      args: [ "tail -f /dev/null" ]
      volumeDevices:
        - name: data
          devicePath: /dev/xvda
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: block-pvc
```

> **Note:**
> When adding a raw block device for a Pod, you specify the device path in the container instead of a mount path.

Binding Block Volumes

If a user requests a raw block volume by indicating this using the `volumeMode` field in the PersistentVolumeClaim spec, the binding rules differ slightly from previous releases that didn't consider this mode as part of the spec. Listed is a table of possible combinations the user and admin might specify for requesting a raw block device. The table indicates if the volume will be bound or not given the combinations: Volume binding matrix for statically provisioned volumes:

PV volumeMode	PVC volumeMode	Result
unspecified	unspecified	BIND
unspecified	Block	NO BIND
unspecified	Filesystem	BIND
Block	unspecified	NO BIND
Block	Block	BIND
Block	Filesystem	NO BIND
Filesystem	Filesystem	BIND
Filesystem	Block	NO BIND
Filesystem	unspecified	BIND

> **Note:**
> Only statically provisioned volumes are supported for alpha release. Administrators should take care to consider these values when working with raw block devices.

Volume Snapshot and Restore Volume from Snapshot Support

ⓘ **FEATURE STATE:** Kubernetes v1.20 [stable]

Volume snapshots only support the out-of-tree CSI volume plugins. For details, see Volume Snapshots. In-tree volume plugins are deprecated. You can read about the deprecated volume plugins in the Volume Plugin FAQ.

Create a PersistentVolumeClaim from a Volume Snapshot

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: restore-pvc
spec:
  storageClassName: csi-hostpath-sc
  dataSource:
    name: new-snapshot-test
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Volume Cloning

Volume Cloning only available for CSI volume plugins.

Create PersistentVolumeClaim from an existing PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: cloned-pvc
spec:
  storageClassName: my-csi-plugin
  dataSource:
    name: existing-src-pvc-name
    kind: PersistentVolumeClaim
```

```
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Volume populators and data sources

Volume cloning and snapshot restore pre-populate a new volume from a built-in *data source*. *Volume populators* extend this mechanism so that a PersistentVolumeClaim can be pre-populated from other kinds of source (a custom resource), referenced through its `dataSourceRef` field:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: populated-pvc
spec:
  dataSourceRef:
    name: example-name
    kind: ExampleDataSource
    apiGroup: example.storage.k8s.io
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

For details, including cross-namespace data sources, see Volume Populators and Data Sources.

Writing Portable Configuration

If you're writing configuration templates or examples that run on a wide range of clusters and need persistent storage, it is recommended that you use the following pattern:

- Include PersistentVolumeClaim objects in your bundle of config (alongside Deployments, ConfigMaps, etc).
- Do not include PersistentVolume objects in the config, since the user instantiating the config may not have permission to create PersistentVolumes.
- Give the user the option of providing a storage class name when instantiating the template.
  - If the user provides a storage class name, put that value into the `persistentVolumeClaim.storageClassName` field. This will cause the PVC to match the right storage class if the cluster has StorageClasses enabled by the admin.
  - If the user does not provide a storage class name, leave the `persistentVolumeClaim.storageClassName` field as nil. This will cause a PV to be automatically provisioned for the user with the default StorageClass in the cluster. Many cluster environments have a default StorageClass installed, or administrators can create their own default StorageClass.
- In your tooling, watch for PVCs that are not getting bound after some time and surface this to the user, as this may indicate that the cluster has no dynamic storage support (in which case the user should create a matching PV) or the cluster has no storage system (in which case the user cannot deploy config requiring PVCs).

What's next

- Learn more about Creating a PersistentVolume.
- Learn more about Creating a PersistentVolumeClaim.
- Read the Persistent Storage design document.

API references

Read about the APIs described in this page:

- `PersistentVolume`
- `PersistentVolumeClaim`

3 - Projected Volumes

This document describes *projected volumes* in Kubernetes. Familiarity with volumes is suggested.

Introduction

A `projected` volume maps several existing volume sources into the same directory.

Currently, the following types of volume sources can be projected:

- `secret`
- `downwardAPI`
- `configMap`
- `serviceAccountToken`
- `clusterTrustBundle`
- `podCertificate`

All sources are required to be in the same namespace as the Pod. For more details, see the all-in-one volume design document.

Example configuration with a secret, a downwardAPI, and a configMap

pods/storage/projected-secret-downwardapi-configmap.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: volume-test
spec:
  containers:
  - name: container-test
    image: busybox:1.28
    command: ["sleep", "3600"]
    volumeMounts:
    - name: all-in-one
      mountPath: "/projected-volume"
      readOnly: true
```

```
  volumes:
  - name: all-in-one
    projected:
      sources:
      - secret:
          name: mysecret
          items:
          - key: username
            path: my-group/my-username
      - downwardAPI:
          items:
          - path: "labels"
            fieldRef:
              fieldPath: metadata.labels
          - path: "cpu_limit"
            resourceFieldRef:
              containerName: container-test
              resource: limits.cpu
      - configMap:
          name: myconfigmap
          items:
          - key: config
            path: my-group/my-config
```

Example configuration: secrets with a non-default permission mode set

pods/storage/projected-secrets-nondefault-permission-mode.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-test
spec:
  containers:
  - name: container-test
    image: busybox:1.28
    command: ["sleep", "3600"]
    volumeMounts:
    - name: all-in-one
      mountPath: "/projected-volume"
```

```
      readOnly: true
  volumes:
  - name: all-in-one
    projected:
      sources:
      - secret:
          name: mysecret
          items:
          - key: username
            path: my-group/my-username
      - secret:
          name: mysecret2
          items:
          - key: password
            path: my-group/my-password
            mode: 0777
```

Each projected volume source is listed in the spec under `sources`. The parameters are nearly the same with two exceptions:

- For secrets, the `secretName` field has been changed to `name` to be consistent with ConfigMap naming.
- The `defaultMode` can only be specified at the projected level and not for each volume source. However, as illustrated above, you can explicitly set the `mode` for each individual projection.

serviceAccountToken projected volumes

You can inject the token for the current service account into a Pod at a specified path. For example:

pods/storage/projected-service-account-token.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: sa-token-test
spec:
  containers:
  - name: container-test
```

```
    image: busybox:1.28
    command: ["sleep", "3600"]
    volumeMounts:
    - name: token-vol
      mountPath: "/service-account"
      readOnly: true
  serviceAccountName: default
  volumes:
  - name: token-vol
    projected:
      sources:
      - serviceAccountToken:
          audience: api
          expirationSeconds: 3600
          path: token
```

The example Pod has a projected volume containing the injected service account token. Containers in this Pod can use that token to access the Kubernetes API server, authenticating with the identity of the pod's ServiceAccount. The `audience` field contains the intended audience of the token. A recipient of the token must identify itself with an identifier specified in the audience of the token, and otherwise should reject the token. This field is optional and it defaults to the identifier of the API server.

The `expirationSeconds` is the expected duration of validity of the service account token. It defaults to 1 hour and must be at least 10 minutes (600 seconds). An administrator can also limit its maximum value by specifying the `--service-account-max-token-expiration` option for the API server. The `path` field specifies a relative path to the mount point of the projected volume.

> **Note:**
> A container using a projected volume source as a `subPath` volume mount will not receive updates for those volume sources.

clusterTrustBundle projected volumes

ⓘ **FEATURE STATE:** Kubernetes v1.33 [beta](disabled by

default)

> **Note:**
> To use this feature in Kubernetes 1.36, you must enable support for ClusterTrustBundle objects with the `ClusterTrustBundle` feature gate and `--runtime-config=certificates.k8s.io/v1beta1/clustertrustbundles=true` kube-apiserver flag, then enable the `ClusterTrustBundleProjection` feature gate.

The `clusterTrustBundle` projected volume source injects the contents of one or more ClusterTrustBundle objects as an automatically-updating file in the container filesystem.

ClusterTrustBundles can be selected either by name or by signer name.

To select by name, use the `name` field to designate a single ClusterTrustBundle object.

To select by signer name, use the `signerName` field (and optionally the `labelSelector` field) to designate a set of ClusterTrustBundle objects that use the given signer name. If `labelSelector` is not present, then all ClusterTrustBundles for that signer are selected.

The kubelet deduplicates the certificates in the selected ClusterTrustBundle objects, normalizes the PEM representations (discarding comments and headers), reorders the certificates, and writes them into the file named by `path`. As the set of selected ClusterTrustBundles or their content changes, kubelet keeps the file up-to-date.

By default, the kubelet will prevent the pod from starting if the named ClusterTrustBundle is not found, or if `signerName` / `labelSelector` do not match any ClusterTrustBundles. If this behavior is not what you want, then set the `optional` field to `true`, and the pod will start up with an empty file at `path`.

pods/storage/projected-clustertrustbundle.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sa-ctb-name-test
spec:
  containers:
  - name: container-test
    image: busybox
    command: ["sleep", "3600"]
```

```
    volumeMounts:
    - name: token-vol
      mountPath: "/root-certificates"
      readOnly: true
  serviceAccountName: default
  volumes:
  - name: token-vol
    projected:
      sources:
      - clusterTrustBundle:
          name: example
          path: example-roots.pem
      - clusterTrustBundle:
          signerName: "example.com/mysigner"
          labelSelector:
            matchLabels:
              version: live
          path: mysigner-roots.pem
          optional: true
```

podCertificate projected volumes

ⓘ **FEATURE STATE:** Kubernetes v1.35 [beta](disabled by default)

> **Note:**
> In Kubernetes 1.36, you must enable support for Pod Certificates using the `PodCertificateRequest` feature gate and the `--runtime-config=certificates.k8s.io/v1beta1/podcertificaterequests=true` kube-apiserver flag.

The `podCertificate` projected volumes source securely provisions a private key and X.509 certificate chain for pod to use as client or server credentials. Kubelet will then handle refreshing the private key and certificate chain when they get close to expiration. The application just has to make sure that it reloads the file promptly when it changes, with a mechanism like `inotify` or polling.

Each `podCertificate` projection supports the following configuration fields:

- `signerName` : The signer you want to issue the certificate. Note that signers may have their own access requirements, and may refuse to issue certificates to your pod.
- `keyType` : The type of private key that should be generated. Valid values are `ED25519` , `ECDSAP256` , `ECDSAP384` , `ECDSAP521` , `RSA3072` , and `RSA4096` .
- `maxExpirationSeconds` : The maximum lifetime you will accept for the certificate issued to the pod. If not set, will be defaulted to `86400` (24 hours). Must be at least `3600` (1 hour), and at most `7862400` (91 days). Kubernetes built-in signers are restricted to a max lifetime of `86400` (1 day). The signer is allowed to issue a certificate with a lifetime shorter than what you've specified.
- `credentialBundlePath` : Relative path within the projection where the credential bundle should be written. The credential bundle is a PEM-formatted file, where the first block is a "PRIVATE KEY" block that contains a PKCS#8-serialized private key, and the remaining blocks are "CERTIFICATE" blocks that comprise the certificate chain (leaf certificate and any intermediates).
- `keyPath` and `certificateChainPath` : Separate paths where Kubelet should write *just* the private key or certificate chain.
- `userAnnotations` : a map that allows you to pass additional information to the signer implementation. It is copied verbatim into the `spec.unverifiedUserAnnotations` field of the PodCertificateRequest objects that Kubelet creates. Entries are subject to the same validation as object metadata annotations, with the addition that all keys must be domain-prefixed. No restrictions are placed on values, except an overall size limitation on the entire field. Other than these basic validations, the API server does not conduct any extra validations. The signer implementations should be very careful when consuming this data. Signers must not inherently trust this data without first performing the appropriate verification steps. Signers should document the keys and values they support. Signers should deny requests that contain keys they do not recognize.

> **Note:**
> Most applications should prefer using `credentialBundlePath` unless they need the key and certificates in separate files for compatibility reasons. Kubelet uses an atomic writing strategy based on symlinks to make sure that when you open the files it projects, you read either the old content or the new content. However, if you read the key and certificate chain from separate files, Kubelet may rotate the credentials after your first read and before your second read, resulting in your application loading a mismatched key and certificate.

pods/storage/projected-podcertificate.yaml

```yaml
# Sample Pod spec that uses a podCertificate projection to request an ED25519
# private key, a certificate from the `coolcert.example.com/foo` signer, and
# write the results to `/var/run/my-x509-credentials/credentialbundle.pem`.
apiVersion: v1
kind: Pod
metadata:
  namespace: default
  name: podcertificate-pod
spec:
  serviceAccountName: default
  containers:
  - image: debian
    name: main
    command: ["sleep", "infinity"]
    volumeMounts:
    - name: my-x509-credentials
      mountPath: /var/run/my-x509-credentials
  volumes:
  - name: my-x509-credentials
    projected:
      defaultMode: 0644
      sources:
      - podCertificate:
          keyType: ED25519
          signerName: coolcert.example.com/foo
          credentialBundlePath: credentialbundle.pem
          userAnnotations:
            example.com/annotation1: "value1"
            example.com/annotation2: "value2"
```

SecurityContext interactions

The proposal for file permission handling in projected service account volume enhancement introduced the projected files having the correct owner permissions set.

Linux

In Linux pods that have a projected volume and `RunAsUser` set in the Pod `SecurityContext`, the projected files have the correct ownership set including container user ownership.

When all containers in a pod have the same `runAsUser` set in their `PodSecurityContext` or container `SecurityContext`, then the kubelet ensures that the contents of the `serviceAccountToken` volume are owned by that user, and the token file has its permission mode set to `0600`.

> **Note:**
> Ephemeral containers added to a Pod after it is created do *not* change volume permissions that were set when the pod was created.
>
> If a Pod's `serviceAccountToken` volume permissions were set to `0600` because all other containers in the Pod have the same `runAsUser`, ephemeral containers must use the same `runAsUser` to be able to read the token.

Windows

In Windows pods that have a projected volume and `RunAsUsername` set in the Pod `SecurityContext`, the ownership is not enforced due to the way user accounts are managed in Windows. Windows stores and manages local user and group accounts in a database file called Security Account Manager (SAM). Each container maintains its own instance of the SAM database, to which the host has no visibility into while the container is running. Windows containers are designed to run the user mode portion of the OS in isolation from the host, hence the maintenance of a virtual SAM database. As a result, the kubelet running on the host does not have the ability to dynamically configure host file ownership for virtualized container accounts. It is recommended that if files on the host machine are to be shared with the container then they should be placed into their own volume mount outside of `C:\`.

By default, the projected files will have the following ownership as shown for an example projected volume file:

```
PS C:\> Get-Acl C:\var\run\secrets\kubernetes.io\serviceaccount\..2021_08_31_22_22_18

Path   : Microsoft.PowerShell.Core\FileSystem::C:\var\run\secrets\kubernetes.io\servi
Owner  : BUILTIN\Administrators
```

```
Group  : NT AUTHORITY\SYSTEM
Access : NT AUTHORITY\SYSTEM Allow  FullControl
         BUILTIN\Administrators Allow  FullControl
         BUILTIN\Users Allow  ReadAndExecute, Synchronize
Audit  :
Sddl   : O:BAG:SYD:AI(A;ID;FA;;;SY)(A;ID;FA;;;BA)(A;ID;0x1200a9;;;BU)
```

This implies all administrator users like `ContainerAdministrator` will have read, write and execute access while, non-administrator users will have read and execute access.

Note:
In general, granting the container access to the host is discouraged as it can open the door for potential security exploits.

Creating a Windows Pod with `RunAsUser` in it's `SecurityContext` will result in the Pod being stuck at `ContainerCreating` forever. So it is advised to not use the Linux only `RunAsUser` option with Windows Pods.

4 - Ephemeral Volumes

This document describes *ephemeral volumes* in Kubernetes. Familiarity with volumes is suggested, in particular PersistentVolumeClaim and PersistentVolume.

Some applications need additional storage but don't care whether that data is stored persistently across restarts. For example, caching services are often limited by memory size and can move infrequently used data into storage that is slower than memory with little impact on overall performance.

Other applications expect some read-only input data to be present in files, like configuration data or secret keys.

Ephemeral volumes are designed for these use cases. Because volumes follow the Pod's lifetime and get created and deleted along with the Pod, Pods can be stopped and restarted without being limited to where some persistent volume is available.

Ephemeral volumes are specified *inline* in the Pod spec, which simplifies application deployment and management.

Types of ephemeral volumes

Kubernetes supports several different kinds of ephemeral volumes for different purposes:

- emptyDir: empty at Pod startup, with storage coming locally from the kubelet base directory (usually the root disk) or RAM
- configMap, downwardAPI, secret: inject different kinds of Kubernetes data into a Pod
- image: allows mounting container image files or artifacts, directly to a Pod.
- CSI ephemeral volumes: similar to the previous volume kinds, but provided by special CSI drivers which specifically support this feature
- generic ephemeral volumes, which can be provided by all storage drivers that also support persistent volumes

`emptyDir`, `configMap`, `downwardAPI`, `secret` are provided as local ephemeral storage. They are managed by kubelet on each node.

CSI ephemeral volumes *must* be provided by third-party CSI storage drivers.

Generic ephemeral volumes *can* be provided by third-party CSI storage drivers, but also by any other storage driver that supports dynamic provisioning. Some CSI drivers are written specifically for CSI ephemeral volumes and do not support dynamic provisioning: those then cannot be used for generic ephemeral volumes.

The advantage of using third-party drivers is that they can offer functionality that Kubernetes itself does not support, for example storage with different performance characteristics than the disk that is managed by kubelet, or injecting different data.

CSI ephemeral volumes

FEATURE STATE: Kubernetes v1.25 [stable]

> **Note:**
> CSI ephemeral volumes are only supported by a subset of CSI drivers. The Kubernetes CSI Drivers list shows which drivers support ephemeral volumes.

Conceptually, CSI ephemeral volumes are similar to `configMap`, `downwardAPI` and `secret` volume types: the storage is managed locally on each node and is created together with other local resources after a Pod has been scheduled onto a node. Kubernetes has no concept of rescheduling Pods anymore at this stage. Volume creation has to be unlikely to fail, otherwise Pod startup gets stuck. In particular, storage capacity aware Pod scheduling is *not* supported for these volumes. They are currently also not covered by the storage resource usage limits of a Pod, because that is something that kubelet can only enforce for storage that it manages itself.

Here's an example manifest for a Pod that uses CSI ephemeral storage:

```yaml
kind: Pod
apiVersion: v1
metadata:
  name: my-csi-app
spec:
  containers:
    - name: my-frontend
      image: busybox:1.28
      volumeMounts:
      - mountPath: "/data"
        name: my-csi-inline-vol
      command: [ "sleep", "1000000" ]
  volumes:
    - name: my-csi-inline-vol
```

```
    csi:
      driver: inline.storage.kubernetes.io
      volumeAttributes:
        foo: bar
```

The `volumeAttributes` determine what volume is prepared by the driver. These attributes are specific to each driver and not standardized. See the documentation of each CSI driver for further instructions.

CSI driver restrictions

CSI ephemeral volumes allow users to provide `volumeAttributes` directly to the CSI driver as part of the Pod spec. A CSI driver allowing `volumeAttributes` that are typically restricted to administrators is NOT suitable for use in an inline ephemeral volume. For example, parameters that are normally defined in the StorageClass should not be exposed to users through the use of inline ephemeral volumes.

Cluster administrators who need to restrict the CSI drivers that are allowed to be used as inline volumes within a Pod spec may do so by:

- Removing `Ephemeral` from `volumeLifecycleModes` in the CSIDriver spec, which prevents the driver from being used as an inline ephemeral volume.
- Using an admission webhook to restrict how this driver is used.

Generic ephemeral volumes

ⓘ **FEATURE STATE:** Kubernetes v1.23 [stable]

Generic ephemeral volumes are similar to `emptyDir` volumes in the sense that they provide a per-pod directory for scratch data that is usually empty after provisioning. But they may also have additional features:

- Storage can be local or network-attached.
- Volumes can have a fixed size that Pods are not able to exceed.
- Volumes may have some initial data, depending on the driver and parameters.
- Typical operations on volumes are supported assuming that the driver supports them, including snapshotting, cloning, resizing, and storage capacity tracking.

Example:

```yaml
kind: Pod
apiVersion: v1
metadata:
  name: my-app
spec:
  containers:
    - name: my-frontend
      image: busybox:1.28
      volumeMounts:
      - mountPath: "/scratch"
        name: scratch-volume
      command: [ "sleep", "1000000" ]
  volumes:
    - name: scratch-volume
      ephemeral:
        volumeClaimTemplate:
          metadata:
            labels:
              type: my-frontend-volume
          spec:
            accessModes: [ "ReadWriteOnce" ]
            storageClassName: "scratch-storage-class"
            resources:
              requests:
                storage: 1Gi
```

Lifecycle and PersistentVolumeClaim

The key design idea is that the parameters for a volume claim are allowed inside a volume source of the Pod. Labels, annotations and the whole set of fields for a PersistentVolumeClaim are supported. When such a Pod gets created, the ephemeral volume controller then creates an actual PersistentVolumeClaim object in the same namespace as the Pod and ensures that the PersistentVolumeClaim gets deleted when the Pod gets deleted.

That triggers volume binding and/or provisioning, either immediately if the StorageClass uses immediate volume binding or when the Pod is tentatively scheduled onto a node (`WaitForFirstConsumer` volume binding mode). The latter is recommended for generic ephemeral volumes because then the scheduler is free to choose a suitable node for the Pod. With immediate binding, the scheduler is forced to select a node that has access to the volume once it is available.

In terms of resource ownership, a Pod that has generic ephemeral storage is the owner of the PersistentVolumeClaim(s) that provide that ephemeral storage. When the Pod is deleted, the Kubernetes garbage collector deletes the PVC, which then usually triggers deletion of the volume because the default reclaim policy of storage classes is to delete volumes. You can create quasi-ephemeral local storage using a StorageClass with a reclaim policy of `retain`: the storage outlives the Pod, and in this case you need to ensure that volume clean up happens separately.

While these PVCs exist, they can be used like any other PVC. In particular, they can be referenced as data source in volume cloning or snapshotting. The PVC object also holds the current status of the volume.

PersistentVolumeClaim naming

Naming of the automatically created PVCs is deterministic: the name is a combination of the Pod name and volume name, with a hyphen (`-`) in the middle. In the example above, the PVC name will be `my-app-scratch-volume`. This deterministic naming makes it easier to interact with the PVC because one does not have to search for it once the Pod name and volume name are known.

The deterministic naming also introduces a potential conflict between different Pods (a Pod "pod-a" with volume "scratch" and another Pod with name "pod" and volume "a-scratch" both end up with the same PVC name "pod-a-scratch") and between Pods and manually created PVCs.

Such conflicts are detected: a PVC is only used for an ephemeral volume if it was created for the Pod. This check is based on the ownership relationship. An existing PVC is not overwritten or modified. But this does not resolve the conflict because without the right PVC, the Pod cannot start.

> **Caution:**
>
> Take care when naming Pods and volumes inside the same namespace, so that these conflicts can't occur.

Security

Using generic ephemeral volumes allows users to create PVCs indirectly if they can create Pods, even if they do not have permission to create PVCs directly. Cluster administrators

must be aware of this. If this does not fit their security model, they should use an admission webhook that rejects objects like Pods that have a generic ephemeral volume.

The normal namespace quota for PVCs still applies, so even if users are allowed to use this new mechanism, they cannot use it to circumvent other policies.

What's next

Ephemeral volumes managed by kubelet

See local ephemeral storage.

CSI ephemeral volumes

- For more information on the design, see the Ephemeral Inline CSI volumes KEP.
- For more information on further development of this feature, see the enhancement tracking issue #596.

Generic ephemeral volumes

- For more information on the design, see the Generic ephemeral inline volumes KEP.

5 - Storage Classes

This document describes the concept of a StorageClass in Kubernetes. Familiarity with volumes and persistent volumes is suggested.

A StorageClass provides a way for administrators to describe the *classes* of storage they offer. Different classes might map to quality-of-service levels, or to backup policies, or to arbitrary policies determined by the cluster administrators. Kubernetes itself is unopinionated about what classes represent.

The Kubernetes concept of a storage class is similar to “profiles” in some other storage system designs.

StorageClass objects

Each StorageClass contains the fields `provisioner`, `parameters`, and `reclaimPolicy`, which are used when a PersistentVolume belonging to the class needs to be dynamically provisioned to satisfy a PersistentVolumeClaim (PVC).

The name of a StorageClass object is significant, and is how users can request a particular class. Administrators set the name and other parameters of a class when first creating StorageClass objects.

As an administrator, you can specify a default StorageClass that applies to any PVCs that don't request a specific class. For more details, see the PersistentVolumeClaim concept.

Here's an example of a StorageClass:

storage/storageclass-low-latency.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: low-latency
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
provisioner: csi-driver.example-vendor.example
reclaimPolicy: Retain # default value is Delete
allowVolumeExpansion: true
mountOptions:
  - discard # this might enable UNMAP / TRIM at the block storage layer
```

```
volumeBindingMode: WaitForFirstConsumer
parameters:
  guaranteedReadWriteLatency: "true" # provider-specific
```

Default StorageClass

You can mark a StorageClass as the default for your cluster. For instructions on setting the default StorageClass, see Change the default StorageClass.

When a PVC does not specify a `storageClassName`, the default StorageClass is used.

If you set the `storageclass.kubernetes.io/is-default-class` annotation to true on more than one StorageClass in your cluster, and you then create a PersistentVolumeClaim with no `storageClassName` set, Kubernetes uses the most recently created default StorageClass.

> **Note:**
> You should try to only have one StorageClass in your cluster that is marked as the default. The reason that Kubernetes allows you to have multiple default StorageClasses is to allow for seamless migration.

You can create a PersistentVolumeClaim without specifying a `storageClassName` for the new PVC, and you can do so even when no default StorageClass exists in your cluster. In this case, the new PVC creates as you defined it, and the `storageClassName` of that PVC remains unset until a default becomes available.

You can have a cluster without any default StorageClass. If you don't mark any StorageClass as default (and one hasn't been set for you by, for example, a cloud provider), then Kubernetes cannot apply that defaulting for PersistentVolumeClaims that need it.

If or when a default StorageClass becomes available, the control plane identifies any existing PVCs without `storageClassName`. For the PVCs that either have an empty value for `storageClassName` or do not have this key, the control plane then updates those PVCs to set `storageClassName` to match the new default StorageClass. If you have an existing PVC where the `storageClassName` is `""`, and you configure a default StorageClass, then this PVC will not get updated.

In order to keep binding to PVs with `storageClassName` set to `""` (while a default StorageClass is present), you need to set the `storageClassName` of the associated PVC to `""`.

Provisioner

Each StorageClass has a provisioner that determines what volume plugin is used for provisioning PVs. This field must be specified.

Volume Plugin	Internal Provisioner	Config Example
AzureFile	✓	Azure File
CephFS	-	-
FC	-	-
FlexVolume	-	-
iSCSI	-	-
Local	-	Local
NFS	-	NFS
PortworxVolume	✓	Portworx Volume
RBD	-	Ceph RBD
VsphereVolume	✓	vSphere

You are not restricted to specifying the "internal" provisioners listed here (whose names are prefixed with "kubernetes.io" and shipped alongside Kubernetes). You can also run and specify external provisioners, which are independent programs that follow a specification defined by Kubernetes. Authors of external provisioners have full discretion over where their code lives, how the provisioner is shipped, how it needs to be run, what volume plugin it uses (including Flex), etc. The repository kubernetes-sigs/sig-storage-lib-external-provisioner houses a library for writing external provisioners that implements the bulk of the specification. Some external provisioners are listed under the repository kubernetes-sigs/sig-storage-lib-external-provisioner.

For example, NFS doesn't provide an internal provisioner, but an external provisioner can be used. There are also cases when 3rd party storage vendors provide their own external

provisioner.

Reclaim policy

PersistentVolumes that are dynamically created by a StorageClass will have the reclaim policy specified in the `reclaimPolicy` field of the class, which can be either `Delete` or `Retain`. If no `reclaimPolicy` is specified when a StorageClass object is created, it will default to `Delete`.

PersistentVolumes that are created manually and managed via a StorageClass will have whatever reclaim policy they were assigned at creation.

Volume expansion

PersistentVolumes can be configured to be expandable. This allows you to resize the volume by editing the corresponding PVC object, requesting a new larger amount of storage.

The following types of volumes support volume expansion, when the underlying StorageClass has the field `allowVolumeExpansion` set to true.

Volume type	Required Kubernetes version for volume expansion
Azure File	1.11
CSI	1.24
FlexVolume	1.13
Portworx	1.11
rbd	1.11

> **Note:**
> You can only use the volume expansion feature to grow a Volume, not to shrink it.

Mount options

PersistentVolumes that are dynamically created by a StorageClass will have the mount options specified in the `mountOptions` field of the class.

If the volume plugin does not support mount options but mount options are specified, provisioning will fail. Mount options are **not** validated on either the class or PV. If a mount option is invalid, the PV mount fails.

Volume binding mode

The `volumeBindingMode` field controls when volume binding and dynamic provisioning should occur. When unset, `Immediate` mode is used by default.

The `Immediate` mode indicates that volume binding and dynamic provisioning occurs once the PersistentVolumeClaim is created. For storage backends that are topology-constrained and not globally accessible from all Nodes in the cluster, PersistentVolumes will be bound or provisioned without knowledge of the Pod's scheduling requirements. This may result in unschedulable Pods.

A cluster administrator can address this issue by specifying the `WaitForFirstConsumer` mode which will delay the binding and provisioning of a PersistentVolume until a Pod using the PersistentVolumeClaim is created. PersistentVolumes will be selected or provisioned conforming to the topology that is specified by the Pod's scheduling constraints. These include, but are not limited to, resource requirements, node selectors, pod affinity and anti-affinity, and taints and tolerations.

The following plugins support `WaitForFirstConsumer` with dynamic provisioning:

- CSI volumes, provided that the specific CSI driver supports this

The following plugins support `WaitForFirstConsumer` with pre-created PersistentVolume binding:

- CSI volumes, provided that the specific CSI driver supports this
- `local`

> **Note:**
> If you choose to use `WaitForFirstConsumer`, do not use `nodeName` in the Pod spec to specify node affinity. If `nodeName` is used in this case, the scheduler will be bypassed and PVC will remain in `pending` state.

> Instead, you can use node selector for `kubernetes.io/hostname`:

storage/storageclass/pod-volume-binding.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: task-pv-pod
spec:
  nodeSelector:
    kubernetes.io/hostname: kube-01
  volumes:
    - name: task-pv-storage
      persistentVolumeClaim:
        claimName: task-pv-claim
  containers:
    - name: task-pv-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: task-pv-storage
```

Allowed topologies

When a cluster operator specifies the `WaitForFirstConsumer` volume binding mode, it is no longer necessary to restrict provisioning to specific topologies in most situations. However, if still required, `allowedTopologies` can be specified.

This example demonstrates how to restrict the topology of provisioned volumes to specific zones and should be used as a replacement for the `zone` and `zones` parameters for the supported plugins.

storage/storageclass/storageclass-topology.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
```

```
metadata:
  name: standard
provisioner: example.com/example
parameters:
  type: pd-standard
volumeBindingMode: WaitForFirstConsumer
allowedTopologies:
- matchLabelExpressions:
  - key: topology.kubernetes.io/zone
    values:
    - us-central-1a
    - us-central-1b
```

Parameters

StorageClasses have parameters that describe volumes belonging to the storage class. Different parameters may be accepted depending on the `provisioner`. When a parameter is omitted, some default is used.

There can be at most 512 parameters defined for a StorageClass. The total length of the parameters object including its keys and values cannot exceed 256 KiB.

AWS EBS

Kubernetes 1.36 does not include a `awsElasticBlockStore` volume type.

The AWSElasticBlockStore in-tree storage driver was deprecated in the Kubernetes v1.19 release and then removed entirely in the v1.27 release.

The Kubernetes project suggests that you use the AWS EBS out-of-tree storage driver instead.

Here is an example StorageClass for the AWS EBS CSI driver:

storage/storageclass/storageclass-aws-ebs.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-sc
provisioner: ebs.csi.aws.com
```

```
volumeBindingMode: WaitForFirstConsumer
parameters:
  csi.storage.k8s.io/fstype: xfs
  type: io1
  iopsPerGB: "50"
  encrypted: "true"
  tagSpecification_1: "key1=value1"
  tagSpecification_2: "key2=value2"
allowedTopologies:
- matchLabelExpressions:
  - key: topology.ebs.csi.aws.com/zone
    values:
    - us-east-2c
```

`tagSpecification` : Tags with this prefix are applied to dynamically provisioned EBS volumes.

AWS EFS

To configure AWS EFS storage, you can use the out-of-tree AWS_EFS_CSI_DRIVER.

storage/storageclass/storageclass-aws-efs.yaml

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: efs-sc
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap
  fileSystemId: fs-92107410
  directoryPerms: "700"
```

- `provisioningMode` : The type of volume to be provisioned by Amazon EFS. Currently, only access point based provisioning is supported (`efs-ap`).
- `fileSystemId` : The file system under which the access point is created.
- `directoryPerms` : The directory permissions of the root directory created by the access point.

For more details, refer to the AWS_EFS_CSI_Driver Dynamic Provisioning documentation.

NFS

To configure NFS storage, you can use the in-tree driver or the NFS CSI driver for Kubernetes (recommended).

storage/storageclass/storageclass-nfs.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: example-nfs
provisioner: example.com/external-nfs
parameters:
  server: nfs-server.example.com
  path: /share
  readOnly: "false"
```

- `server` : Server is the hostname or IP address of the NFS server.
- `path` : Path that is exported by the NFS server.
- `readOnly` : A flag indicating whether the storage will be mounted as read only (default false).

Kubernetes doesn't include an internal NFS provisioner. You need to use an external provisioner to create a StorageClass for NFS. Here are some examples:

- NFS Ganesha server and external provisioner
- NFS subdir external provisioner

vSphere

There are two types of provisioners for vSphere storage classes:

- CSI provisioner: `csi.vsphere.vmware.com`
- vCP provisioner: `kubernetes.io/vsphere-volume`

In-tree provisioners are deprecated. For more information on the CSI provisioner, see Kubernetes vSphere CSI Driver and vSphereVolume CSI migration.

CSI Provisioner

The vSphere CSI StorageClass provisioner works with Tanzu Kubernetes clusters. For an example, refer to the vSphere CSI repository.

vCP Provisioner

The following examples use the VMware Cloud Provider (vCP) StorageClass provisioner.

1. Create a StorageClass with a user specified disk format.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/vsphere-volume
parameters:
  diskformat: zeroedthick
```

`diskformat`: `thin`, `zeroedthick` and `eagerzeroedthick`. Default: `"thin"`.

2. Create a StorageClass with a disk format on a user specified datastore.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/vsphere-volume
parameters:
  diskformat: zeroedthick
  datastore: VSANDatastore
```

`datastore`: The user can also specify the datastore in the StorageClass. The volume will be created on the datastore specified in the StorageClass, which in this case is `VSANDatastore`. This field is optional. If the datastore is not specified, then the volume will be created on the datastore specified in the vSphere config file used to initialize the vSphere Cloud Provider.

3. Storage Policy Management inside kubernetes

   - Using existing vCenter SPBM policy

     One of the most important features of vSphere for Storage Management is policy based Management. Storage Policy Based Management (SPBM) is a storage policy framework that provides a single unified control plane across a broad range of data services and storage solutions. SPBM enables vSphere administrators to overcome upfront storage provisioning challenges, such as capacity planning, differentiated service levels and managing capacity headroom.

     The SPBM policies can be specified in the StorageClass using the `storagePolicyName` parameter.

   - Virtual SAN policy support inside Kubernetes

     Vsphere Infrastructure (VI) Admins will have the ability to specify custom Virtual SAN Storage Capabilities during dynamic volume provisioning. You can now define storage requirements, such as performance and availability, in the form of storage capabilities during dynamic volume provisioning. The storage capability requirements are converted into a Virtual SAN policy which are then pushed down to the Virtual SAN layer when a persistent volume (virtual disk) is being created. The virtual disk is distributed across the Virtual SAN datastore to meet the requirements.

     You can see Storage Policy Based Management for dynamic provisioning of volumes for more details on how to use storage policies for persistent volumes management.

Ceph RBD (deprecated)

> **Note:**
>
> ⓘ **FEATURE STATE:** Kubernetes v1.28 [deprecated]
>
> This internal provisioner of Ceph RBD is deprecated. Please use CephFS RBD CSI driver.

`storage/storageclass/storageclass-ceph-rbd.yaml`

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/rbd # This provisioner is deprecated
parameters:
  monitors: 198.19.254.105:6789
  adminId: kube
  adminSecretName: ceph-secret
  adminSecretNamespace: kube-system
  pool: kube
  userId: kube
  userSecretName: ceph-secret-user
  userSecretNamespace: default
  fsType: ext4
  imageFormat: "2"
  imageFeatures: "layering"
```

- `monitors` : Ceph monitors, comma delimited. This parameter is required.
- `adminId` : Ceph client ID that is capable of creating images in the pool. Default is "admin".
- `adminSecretName` : Secret Name for `adminId` . This parameter is required. The provided secret must have type "kubernetes.io/rbd".
- `adminSecretNamespace` : The namespace for `adminSecretName` . Default is "default".
- `pool` : Ceph RBD pool. Default is "rbd".
- `userId` : Ceph client ID that is used to map the RBD image. Default is the same as `adminId` .
- `userSecretName` : The name of Ceph Secret for `userId` to map RBD image. It must exist in the same namespace as PVCs. This parameter is required. The provided secret must have type "kubernetes.io/rbd", for example created in this way:

```
kubectl create secret generic ceph-secret --type="kubernetes.io/rbd" \
  --from-literal=key='QVFEQ1pMdFhPUnQrSmhBQUFYaERWNHJsZ3BsMmNjcDR6RFZST0E9PQ==' \
  --namespace=kube-system
```

- `userSecretNamespace` : The namespace for `userSecretName` .
- `fsType` : fsType that is supported by kubernetes. Default: `"ext4"` .
- `imageFormat` : Ceph RBD image format, "1" or "2". Default is "2".
- `imageFeatures` : This parameter is optional and should only be used if you set `imageFormat` to "2". Currently supported features are `layering` only. Default is "", and no features are turned on.

Azure Disk

Kubernetes 1.36 does not include a `azureDisk` volume type.

The `azureDisk` in-tree storage driver was deprecated in the Kubernetes v1.19 release and then removed entirely in the v1.27 release.

The Kubernetes project suggests that you use the Azure Disk third party storage driver instead.

Azure File (deprecated)

storage/storageclass/storageclass-azure-file.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: azurefile
provisioner: kubernetes.io/azure-file
parameters:
  skuName: Standard_LRS
  location: eastus
  storageAccount: azure_storage_account_name # example value
```

- `skuName` : Azure storage account SKU tier. Default is empty.
- `location` : Azure storage account location. Default is empty.
- `storageAccount` : Azure storage account name. Default is empty. If a storage account is not provided, all storage accounts associated with the resource group are searched to find one that matches `skuName` and `location` . If a storage account is provided, it must

reside in the same resource group as the cluster, and `skuName` and `location` are ignored.

- `secretNamespace` : the namespace of the secret that contains the Azure Storage Account Name and Key. Default is the same as the Pod.
- `secretName` : the name of the secret that contains the Azure Storage Account Name and Key. Default is `azure-storage-account-<accountName>-secret`
- `readOnly` : a flag indicating whether the storage will be mounted as read only. Defaults to false which means a read/write mount. This setting will impact the `ReadOnly` setting in VolumeMounts as well.

During storage provisioning, a secret named by `secretName` is created for the mounting credentials. If the cluster has enabled both RBAC and Controller Roles, add the `create` permission of resource `secret` for clusterrole `system:controller:persistent-volume-binder` .

In a multi-tenancy context, it is strongly recommended to set the value for `secretNamespace` explicitly, otherwise the storage account credentials may be read by other users.

Portworx volume (deprecated)

storage/storageclass/storageclass-portworx-volume.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: portworx-io-priority-high
provisioner: kubernetes.io/portworx-volume # This provisioner is deprecated
parameters:
  repl: "1"
  snap_interval: "70"
  priority_io: "high"
```

- `fs` : filesystem to be laid out: `none/xfs/ext4` (default: `ext4`).
- `block_size` : block size in Kbytes (default: `32`).
- `repl` : number of synchronous replicas to be provided in the form of replication factor `1..3` (default: `1`) A string is expected here i.e. `"1"` and not `1` .
- `priority_io` : determines whether the volume will be created from higher performance or a lower priority storage `high/medium/low` (default: `low`).

- `snap_interval` : clock/time interval in minutes for when to trigger snapshots. Snapshots are incremental based on difference with the prior snapshot, 0 disables snaps (default: `0`). A string is expected here i.e. `"70"` and not `70` .
- `aggregation_level` : specifies the number of chunks the volume would be distributed into, 0 indicates a non-aggregated volume (default: `0`). A string is expected here i.e. `"0"` and not `0`
- `ephemeral` : specifies whether the volume should be cleaned-up after unmount or should be persistent. `emptyDir` use case can set this value to true and `persistent volumes` use case such as for databases like Cassandra should set to false, `true/false` (default `false`). A string is expected here i.e. `"true"` and not `true` .

Local

storage/storageclass/storageclass-local.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-storage
provisioner: kubernetes.io/no-provisioner # indicates that this StorageClass does not
volumeBindingMode: WaitForFirstConsumer
```

Local volumes do not support dynamic provisioning in Kubernetes 1.36; however a StorageClass should still be created to delay volume binding until a Pod is actually scheduled to the appropriate node. This is specified by the `WaitForFirstConsumer` volume binding mode.

Delaying volume binding allows the scheduler to consider all of a Pod's scheduling constraints when choosing an appropriate PersistentVolume for a PersistentVolumeClaim.

6 - Volume Attributes Classes

ⓘ **FEATURE STATE:** Kubernetes v1.36 [stable](enabled by default)

This page assumes that you are familiar with StorageClasses, volumes and PersistentVolumes in Kubernetes.

A VolumeAttributesClass provides a way for administrators to describe the mutable "classes" of storage they offer. Different classes might map to different quality-of-service levels. Kubernetes itself is un-opinionated about what these classes represent.

This feature is generally available (GA) as of version 1.34, and users have the option to disable it.

You can also only use VolumeAttributesClasses with storage backed by Container Storage Interface, and only where the relevant CSI driver implements the `ModifyVolume` API.

The VolumeAttributesClass API

Each VolumeAttributesClass contains the `driverName` and `parameters`, which are used when a PersistentVolume (PV) belonging to the class needs to be dynamically provisioned or modified.

The name of a VolumeAttributesClass object is significant and is how users can request a particular class. Administrators set the name and other parameters of a class when first creating VolumeAttributesClass objects. While the name of a VolumeAttributesClass object in a `PersistentVolumeClaim` is mutable, the parameters in an existing class are immutable.

```yaml
apiVersion: storage.k8s.io/v1
kind: VolumeAttributesClass
metadata:
  name: silver
driverName: pd.csi.storage.gke.io
parameters:
  provisioned-iops: "3000"
  provisioned-throughput: "50"
```

Provisioner

Each VolumeAttributesClass has a provisioner that determines what volume plugin is used for provisioning PVs. The field `driverName` must be specified.

The feature support for VolumeAttributesClass is implemented in kubernetes-csi/external-provisioner.

You are not restricted to specifying the kubernetes-csi/external-provisioner. You can also run and specify external provisioners, which are independent programs that follow a specification defined by Kubernetes. Authors of external provisioners have full discretion over where their code lives, how the provisioner is shipped, how it needs to be run, what volume plugin it uses, etc.

To understand how the provisioner works with VolumeAttributesClass, refer to the CSI external-provisioner documentation.

Resizer

Each VolumeAttributesClass has a resizer that determines what volume plugin is used for modifying PVs. The field `driverName` must be specified.

The modifying volume feature support for VolumeAttributesClass is implemented in kubernetes-csi/external-resizer.

For example, an existing PersistentVolumeClaim is using a VolumeAttributesClass named silver:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-pv-claim
spec:
  …
  volumeAttributesClassName: silver
  …
```

A new VolumeAttributesClass gold is available in the cluster:

```
apiVersion: storage.k8s.io/v1
```

```yaml
kind: VolumeAttributesClass
metadata:
  name: gold
driverName: pd.csi.storage.gke.io
parameters:
  iops: "4000"
  throughput: "60"
```

The end user can update the PVC with the new VolumeAttributesClass gold and apply:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-pv-claim
spec:
  …
  volumeAttributesClassName: gold
  …
```

To understand how the resizer works with VolumeAttributesClass, refer to the CSI external-resizer documentation.

Parameters

VolumeAttributeClasses have parameters that describe volumes belonging to them. Different parameters may be accepted depending on the provisioner or the resizer. For example, the value `4000`, for the parameter `iops`, and the parameter `throughput` are specific to GCE PD. When a parameter is omitted, the default is used at volume provisioning. If a user applies the PVC with a different VolumeAttributesClass with omitted parameters, the default value of the parameters may be used depending on the CSI driver implementation. Please refer to the related CSI driver documentation for more details.

There can be at most 512 parameters defined for a VolumeAttributesClass. The total length of the parameters object including its keys and values cannot exceed 256 KiB.

7 - Dynamic Volume Provisioning

Dynamic volume provisioning allows storage volumes to be created on-demand. Without dynamic provisioning, cluster administrators have to manually make calls to their cloud or storage provider to create new storage volumes, and then create `PersistentVolume` objects to represent them in Kubernetes. The dynamic provisioning feature eliminates the need for cluster administrators to pre-provision storage. Instead, it automatically provisions storage when users create `PersistentVolumeClaim` objects.

Background

The implementation of dynamic volume provisioning is based on the API object `StorageClass` from the API group `storage.k8s.io`. A cluster administrator can define as many `StorageClass` objects as needed, each specifying a *volume plugin* (aka *provisioner*) that provisions a volume and the set of parameters to pass to that provisioner when provisioning. A cluster administrator can define and expose multiple flavors of storage (from the same or different storage systems) within a cluster, each with a custom set of parameters. This design also ensures that end users don't have to worry about the complexity and nuances of how storage is provisioned, but still have the ability to select from multiple storage options.

For more details, see the Storage Classes concept.

Enabling Dynamic Provisioning

To enable dynamic provisioning, a cluster administrator needs to pre-create one or more StorageClass objects for users. StorageClass objects define which provisioner should be used and what parameters should be passed to that provisioner when dynamic provisioning is invoked. The name of a StorageClass object must be a valid DNS subdomain name.

The following manifest creates a storage class "slow" which provisions standard disk-like persistent disks.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: slow
```

```
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
```

The following manifest creates a storage class "fast" which provisions SSD-like persistent disks.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
```

Using Dynamic Provisioning

Users request dynamically provisioned storage by including a storage class in their `PersistentVolumeClaim`. Before Kubernetes v1.6, this was done via the `volume.beta.kubernetes.io/storage-class` annotation. However, this annotation is deprecated since v1.9. Users now can and should instead use the `storageClassName` field of the `PersistentVolumeClaim` object. The value of this field must match the name of a `StorageClass` configured by the administrator (see Enabling Dynamic Provisioning).

To select the "fast" storage class, for example, a user would create the following PersistentVolumeClaim:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: claim1
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast
  resources:
    requests:
```

```
      storage: 30Gi
```

This claim results in an SSD-like Persistent Disk being automatically provisioned. When the claim is deleted, the volume is destroyed.

Defaulting Behavior

Dynamic provisioning can be enabled on a cluster such that all claims are dynamically provisioned if no storage class is specified. A cluster administrator can enable this behavior by:

- Marking one `StorageClass` object as *default*.
- Making sure that the `DefaultStorageClass` admission controller is enabled on the API server.

An administrator can mark a specific `StorageClass` as default by adding the `storageclass.kubernetes.io/is-default-class` annotation to it. When a default `StorageClass` exists in a cluster and a user creates a `PersistentVolumeClaim` with `storageClassName` unspecified, the `DefaultStorageClass` admission controller automatically adds the `storageClassName` field pointing to the default storage class.

Note that if you set the `storageclass.kubernetes.io/is-default-class` annotation to true on more than one StorageClass in your cluster, and you then create a `PersistentVolumeClaim` with no `storageClassName` set, Kubernetes uses the most recently created default StorageClass.

Topology Awareness

In Multi-Zone clusters, Pods can be spread across Zones in a Region. Single-Zone storage backends should be provisioned in the Zones where Pods are scheduled. This can be accomplished by setting the Volume Binding Mode.

8 - Volume Snapshots

In Kubernetes, a *VolumeSnapshot* represents a snapshot of a volume on a storage system. This document assumes that you are already familiar with Kubernetes persistent volumes.

Introduction

Similar to how API resources `PersistentVolume` and `PersistentVolumeClaim` are used to provision volumes for users and administrators, `VolumeSnapshotContent` and `VolumeSnapshot` API resources are provided to create volume snapshots for users and administrators.

A `VolumeSnapshotContent` is a snapshot taken from a volume in the cluster that has been provisioned by an administrator. It is a resource in the cluster just like a PersistentVolume is a cluster resource.

A `VolumeSnapshot` is a request for snapshot of a volume by a user. It is similar to a PersistentVolumeClaim.

`VolumeSnapshotClass` allows you to specify different attributes belonging to a `VolumeSnapshot`. These attributes may differ among snapshots taken from the same volume on the storage system and therefore cannot be expressed by using the same `StorageClass` of a `PersistentVolumeClaim`.

Volume snapshots provide Kubernetes users with a standardized way to copy a volume's contents at a particular point in time without creating an entirely new volume. This functionality enables, for example, database administrators to backup databases before performing edit or delete modifications.

Users need to be aware of the following when using this feature:

- API Objects `VolumeSnapshot`, `VolumeSnapshotContent`, and `VolumeSnapshotClass` are CRDs, not part of the core API.
- `VolumeSnapshot` support is only available for CSI drivers.
- As part of the deployment process of `VolumeSnapshot`, the Kubernetes team provides a snapshot controller to be deployed into the control plane, and a sidecar helper container called csi-snapshotter to be deployed together with the CSI driver. The snapshot controller watches `VolumeSnapshot` and `VolumeSnapshotContent` objects and is responsible for the creation and deletion of `VolumeSnapshotContent` object. The sidecar csi-snapshotter watches `VolumeSnapshotContent` objects and triggers `CreateSnapshot` and `DeleteSnapshot` operations against a CSI endpoint.

- There is also a validating webhook server which provides tightened validation on snapshot objects. This should be installed by the Kubernetes distros along with the snapshot controller and CRDs, not CSI drivers. It should be installed in all Kubernetes clusters that has the snapshot feature enabled.
- CSI drivers may or may not have implemented the volume snapshot functionality. The CSI drivers that have provided support for volume snapshot will likely use the csi-snapshotter. See CSI Driver documentation for details.
- The CRDs and snapshot controller installations are the responsibility of the Kubernetes distribution.

For advanced use cases, such as creating group snapshots of multiple volumes, see the external CSI Volume Group Snapshot documentation.

Lifecycle of a volume snapshot and volume snapshot content

`VolumeSnapshotContents` are resources in the cluster. `VolumeSnapshots` are requests for those resources. The interaction between `VolumeSnapshotContents` and `VolumeSnapshots` follow this lifecycle:

Provisioning Volume Snapshot

There are two ways snapshots may be provisioned: pre-provisioned or dynamically provisioned.

Pre-provisioned

A cluster administrator creates a number of `VolumeSnapshotContents`. They carry the details of the real volume snapshot on the storage system which is available for use by cluster users. They exist in the Kubernetes API and are available for consumption.

Dynamic

Instead of using a pre-existing snapshot, you can request that a snapshot to be dynamically taken from a PersistentVolumeClaim. The VolumeSnapshotClass specifies storage provider-specific parameters to use when taking a snapshot.

Binding

The snapshot controller handles the binding of a `VolumeSnapshot` object with an appropriate `VolumeSnapshotContent` object, in both pre-provisioned and dynamically provisioned scenarios. The binding is a one-to-one mapping.

In the case of pre-provisioned binding, the VolumeSnapshot will remain unbound until the requested VolumeSnapshotContent object is created.

Persistent Volume Claim as Snapshot Source Protection

The purpose of this protection is to ensure that in-use PersistentVolumeClaim API objects are not removed from the system while a snapshot is being taken from it (as this may result in data loss).

While a snapshot is being taken of a PersistentVolumeClaim, that PersistentVolumeClaim is in-use. If you delete a PersistentVolumeClaim API object in active use as a snapshot source, the PersistentVolumeClaim object is not removed immediately. Instead, removal of the PersistentVolumeClaim object is postponed until the snapshot is readyToUse or aborted.

Delete

Deletion is triggered by deleting the `VolumeSnapshot` object, and the `DeletionPolicy` will be followed. If the `DeletionPolicy` is `Delete`, then the underlying storage snapshot will be deleted along with the `VolumeSnapshotContent` object. If the `DeletionPolicy` is `Retain`, then both the underlying snapshot and `VolumeSnapshotContent` remain.

VolumeSnapshots

Each VolumeSnapshot contains a spec and a status.

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: new-snapshot-test
spec:
  volumeSnapshotClassName: csi-hostpath-snapclass
  source:
    persistentVolumeClaimName: pvc-test
```

`persistentVolumeClaimName` is the name of the PersistentVolumeClaim data source for the snapshot. This field is required for dynamically provisioning a snapshot.

A volume snapshot can request a particular class by specifying the name of a VolumeSnapshotClass using the attribute `volumeSnapshotClassName`. If nothing is set, then the default class is used if available.

For pre-provisioned snapshots, you need to specify a `volumeSnapshotContentName` as the source for the snapshot as shown in the following example. The `volumeSnapshotContentName` source field is required for pre-provisioned snapshots.

```yaml
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: test-snapshot
spec:
  source:
    volumeSnapshotContentName: test-content
```

Volume Snapshot Contents

Each VolumeSnapshotContent contains a spec and status. In dynamic provisioning, the snapshot common controller creates `VolumeSnapshotContent` objects. Here is an example:

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotContent
metadata:
  name: snapcontent-72d9a349-aacd-42d2-a240-d775650d2455
spec:
  deletionPolicy: Delete
  driver: hostpath.csi.k8s.io
  source:
    volumeHandle: ee0cfb94-f8d4-11e9-b2d8-0242ac110002
  sourceVolumeMode: Filesystem
  volumeSnapshotClassName: csi-hostpath-snapclass
  volumeSnapshotRef:
    name: new-snapshot-test
    namespace: default
    uid: 72d9a349-aacd-42d2-a240-d775650d2455
```

`volumeHandle` is the unique identifier of the volume created on the storage backend and returned by the CSI driver during the volume creation. This field is required for dynamically provisioning a snapshot. It specifies the volume source of the snapshot.

For pre-provisioned snapshots, you (as cluster administrator) are responsible for creating the `VolumeSnapshotContent` object as follows.

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotContent
metadata:
  name: new-snapshot-content-test
spec:
  deletionPolicy: Delete
  driver: hostpath.csi.k8s.io
  source:
    snapshotHandle: 7bdd0de3-aaeb-11e8-9aae-0242ac110002
  sourceVolumeMode: Filesystem
  volumeSnapshotRef:
    name: new-snapshot-test
    namespace: default
```

`snapshotHandle` is the unique identifier of the volume snapshot created on the storage backend. This field is required for the pre-provisioned snapshots. It specifies the CSI

snapshot id on the storage system that this `VolumeSnapshotContent` represents.

`sourceVolumeMode` is the mode of the volume whose snapshot is taken. The value of the `sourceVolumeMode` field can be either `Filesystem` or `Block`. If the source volume mode is not specified, Kubernetes treats the snapshot as if the source volume's mode is unknown.

`volumeSnapshotRef` is the reference of the corresponding `VolumeSnapshot`. Note that when the `VolumeSnapshotContent` is being created as a pre-provisioned snapshot, the `VolumeSnapshot` referenced in `volumeSnapshotRef` might not exist yet.

Converting the volume mode of a Snapshot

If the `VolumeSnapshots` API installed on your cluster supports the `sourceVolumeMode` field, then the API has the capability to prevent unauthorized users from converting the mode of a volume.

To check if your cluster has capability for this feature, run the following command:

```
$ kubectl get crd volumesnapshotcontent -o yaml
```

If you want to allow users to create a `PersistentVolumeClaim` from an existing `VolumeSnapshot`, but with a different volume mode than the source, the annotation `snapshot.storage.kubernetes.io/allow-volume-mode-change: "true"` needs to be added to the `VolumeSnapshotContent` that corresponds to the `VolumeSnapshot`.

For pre-provisioned snapshots, `spec.sourceVolumeMode` needs to be populated by the cluster administrator.

An example `VolumeSnapshotContent` resource with this feature enabled would look like:

```yaml
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotContent
metadata:
  name: new-snapshot-content-test
  annotations:
    - snapshot.storage.kubernetes.io/allow-volume-mode-change: "true"
spec:
  deletionPolicy: Delete
  driver: hostpath.csi.k8s.io
```

```
  source:
    snapshotHandle: 7bdd0de3-aaeb-11e8-9aae-0242ac110002
  sourceVolumeMode: Filesystem
  volumeSnapshotRef:
    name: new-snapshot-test
    namespace: default
```

Provisioning Volumes from Snapshots

You can provision a new volume, pre-populated with data from a snapshot, by using the *dataSource* field in the `PersistentVolumeClaim` object.

For more details, see Volume Snapshot and Restore Volume from Snapshot.

9 - Volume Snapshot Classes

This document describes the concept of VolumeSnapshotClass in Kubernetes. Familiarity with volume snapshots and storage classes is suggested.

Introduction

Just like StorageClass provides a way for administrators to describe the "classes" of storage they offer when provisioning a volume, VolumeSnapshotClass provides a way to describe the "classes" of storage when provisioning a volume snapshot.

The VolumeSnapshotClass Resource

Each VolumeSnapshotClass contains the fields `driver`, `deletionPolicy`, and `parameters`, which are used when a VolumeSnapshot belonging to the class needs to be dynamically provisioned.

The name of a VolumeSnapshotClass object is significant, and is how users can request a particular class. Administrators set the name and other parameters of a class when first creating VolumeSnapshotClass objects, and the objects cannot be updated once they are created.

> **Note:**
> Installation of the CRDs is the responsibility of the Kubernetes distribution. Without the required CRDs present, the creation of a VolumeSnapshotClass fails.

```yaml
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-hostpath-snapclass
driver: hostpath.csi.k8s.io
deletionPolicy: Delete
parameters:
```

Administrators can specify a default VolumeSnapshotClass for VolumeSnapshots that don't request any particular class to bind to by adding the `snapshot.storage.kubernetes.io/is-default-class: "true"` annotation:

```yaml
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-hostpath-snapclass
  annotations:
    snapshot.storage.kubernetes.io/is-default-class: "true"
driver: hostpath.csi.k8s.io
deletionPolicy: Delete
parameters:
```

If multiple CSI drivers exist, a default VolumeSnapshotClass can be specified for each of them.

VolumeSnapshotClass dependencies

When you create a VolumeSnapshot without specifying a VolumeSnapshotClass, Kubernetes automatically selects a default VolumeSnapshotClass that has a CSI driver matching the CSI driver of the PVC's StorageClass.

This behavior allows multiple default VolumeSnapshotClass objects to coexist in a cluster, as long as each one is associated with a unique CSI driver.

Always ensure that there is only one default VolumeSnapshotClass for each CSI driver. If multiple default VolumeSnapshotClass objects are created using the same CSI driver, a VolumeSnapshot creation will fail because Kubernetes cannot determine which one to use.

Driver

Volume snapshot classes have a driver that determines what CSI volume plugin is used for provisioning VolumeSnapshots. This field must be specified.

DeletionPolicy

Volume snapshot classes have a deletionPolicy. It enables you to configure what happens to a VolumeSnapshotContent when the VolumeSnapshot object it is bound to is to be deleted. The deletionPolicy of a volume snapshot class can either be `Retain` or `Delete`. This field must be specified.

If the deletionPolicy is `Delete`, then the underlying storage snapshot will be deleted along with the VolumeSnapshotContent object. If the deletionPolicy is `Retain`, then both the underlying snapshot and VolumeSnapshotContent remain.

Parameters

Volume snapshot classes have parameters that describe volume snapshots belonging to the volume snapshot class. Different parameters may be accepted depending on the `driver`.

10 - CSI Volume Cloning

This document describes the concept of cloning existing CSI Volumes in Kubernetes. Familiarity with Volumes is suggested.

Introduction

The CSI Volume Cloning feature adds support for specifying existing PVCs in the `dataSource` field to indicate a user would like to clone a Volume.

A Clone is defined as a duplicate of an existing Kubernetes Volume that can be consumed as any standard Volume would be. The only difference is that upon provisioning, rather than creating a "new" empty Volume, the back end device creates an exact duplicate of the specified Volume.

The implementation of cloning, from the perspective of the Kubernetes API, adds the ability to specify an existing PVC as a dataSource during new PVC creation. The source PVC must be bound and available (not in use).

Users need to be aware of the following when using this feature:

- Cloning support (`VolumePVCDataSource`) is only available for CSI drivers.
- Cloning support is only available for dynamic provisioners.
- CSI drivers may or may not have implemented the volume cloning functionality.
- You can only clone a PVC when it exists in the same namespace as the destination PVC (source and destination must be in the same namespace).
- Cloning is supported with a different Storage Class.
  - Destination volume can be the same or a different storage class as the source.
  - Default storage class can be used and storageClassName omitted in the spec.
- Cloning can only be performed between two volumes that use the same VolumeMode setting (if you request a block mode volume, the source MUST also be block mode)

Provisioning

Clones are provisioned like any other PVC with the exception of adding a dataSource that references an existing PVC in the same namespace.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
    name: clone-of-pvc-1
    namespace: myns
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: cloning
  resources:
    requests:
      storage: 5Gi
  dataSource:
    kind: PersistentVolumeClaim
    name: pvc-1
```

> **Note:**
> You must specify a capacity value for `spec.resources.requests.storage`, and the value you specify must be the same or larger than the capacity of the source volume.

The result is a new PVC with the name `clone-of-pvc-1` that has the exact same content as the specified source `pvc-1`.

Usage

Upon availability of the new PVC, the cloned PVC is consumed the same as other PVC. It's also expected at this point that the newly created PVC is an independent object. It can be consumed, cloned, snapshotted, or deleted independently and without consideration for it's original dataSource PVC. This also implies that the source is not linked in any way to the newly created clone, it may also be modified or deleted without affecting the newly created clone.

11 - Volume Populators and Data Sources

This document describes *volume populators* and *data sources* in Kubernetes. Familiarity with persistent volumes is suggested.

When you create a PersistentVolumeClaim, the volume that Kubernetes provisions for it normally starts empty. A *data source* lets you instead request that the new volume be pre-populated with existing data. *Volume populators* are the controllers that carry out that population, based on the data source that the PersistentVolumeClaim references.

Kubernetes has built-in support for data sources that clone an existing volume or that restore a volume snapshot. Custom volume populators extend this mechanism. The data source is a custom resource, that is, an object whose type is defined by a CustomResourceDefinition. A populator controller watches for PersistentVolumeClaims that reference such a resource and fills the new volume from it.

Volume populators and data sources

ⓘ **FEATURE STATE:** Kubernetes v1.24 [beta]

Kubernetes supports custom volume populators. To use custom volume populators, you must enable the `AnyVolumeDataSource` feature gate for the kube-apiserver and kube-controller-manager.

Volume populators take advantage of a PVC spec field called `dataSourceRef`. Unlike the `dataSource` field, which can only contain either a reference to another PersistentVolumeClaim or to a VolumeSnapshot, the `dataSourceRef` field can contain a reference to any object in the same namespace, except for core objects other than PVCs. For clusters that have the feature gate enabled, use of the `dataSourceRef` is preferred over `dataSource`.

Data source references

The `dataSourceRef` field behaves almost the same as the `dataSource` field. If one is specified while the other is not, the API server will give both fields the same value. Neither field can be changed after creation, and attempting to specify different values for the two

fields will result in a validation error. Therefore the two fields will always have the same contents.

There are two differences between the `dataSourceRef` field and the `dataSource` field that users should be aware of:

- The `dataSource` field ignores invalid values (as if the field was blank) while the `dataSourceRef` field never ignores values and will cause an error if an invalid value is used. Invalid values are any core object (objects with no apiGroup) except for PVCs.
- The `dataSourceRef` field may contain different types of objects, while the `dataSource` field only allows PVCs and VolumeSnapshots.

When the `CrossNamespaceVolumeDataSource` feature is enabled, there are additional differences:

- The `dataSource` field only allows local objects, while the `dataSourceRef` field allows objects in any namespaces.
- When namespace is specified, `dataSource` and `dataSourceRef` are not synced.

Users should always use `dataSourceRef` on clusters that have the feature gate enabled, and fall back to `dataSource` on clusters that do not. It is not necessary to look at both fields under any circumstance. The duplicated values with slightly different semantics exist only for backwards compatibility. In particular, a mixture of older and newer controllers are able to interoperate because the fields are the same.

Using volume populators

Volume populators are controllers that can create non-empty volumes, where the contents of the volume are determined by a Custom Resource. Users create a populated volume by referring to a Custom Resource using the `dataSourceRef` field:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: populated-pvc
spec:
  dataSourceRef:
    name: example-name
    kind: ExampleDataSource
    apiGroup: example.storage.k8s.io
  accessModes:
```

```
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Because volume populators are external components, attempts to create a PVC that uses one can fail if not all the correct components are installed. External controllers should generate events on the PVC to provide feedback on the status of the creation, including warnings if the PVC cannot be created due to some missing component.

You can install the alpha volume data source validator controller into your cluster. That controller generates warning Events on a PVC in the case that no populator is registered to handle that kind of data source. When a suitable populator is installed for a PVC, it's the responsibility of that populator controller to report Events that relate to volume creation and issues during the process.

Cross namespace data sources

FEATURE STATE: Kubernetes v1.26 [alpha]

Kubernetes supports cross namespace volume data sources. To use cross namespace volume data sources, you must enable the `AnyVolumeDataSource` and `CrossNamespaceVolumeDataSource` feature gates for the kube-apiserver and kube-controller-manager. Also, you must enable the `CrossNamespaceVolumeDataSource` feature gate for the csi-provisioner.

Enabling the `CrossNamespaceVolumeDataSource` feature gate allows you to specify a namespace in the dataSourceRef field.

> **Note:**
> When you specify a namespace for a volume data source, Kubernetes checks for a ReferenceGrant in the other namespace before accepting the reference. ReferenceGrant is part of the `gateway.networking.k8s.io` extension APIs. See ReferenceGrant in the Gateway API documentation for details. This means that you must extend your Kubernetes cluster with at least ReferenceGrant from the Gateway API before you can use this mechanism.

Using a cross-namespace volume data source

FEATURE STATE: Kubernetes v1.26 [alpha]

Create a ReferenceGrant to allow the namespace owner to accept the reference. You define a populated volume by specifying a cross namespace volume data source using the `dataSourceRef` field. You must already have a valid ReferenceGrant in the source namespace:

```yaml
apiVersion: gateway.networking.k8s.io/v1beta1
kind: ReferenceGrant
metadata:
  name: allow-ns1-pvc
  namespace: default
spec:
  from:
  - group: ""
    kind: PersistentVolumeClaim
    namespace: ns1
  to:
  - group: snapshot.storage.k8s.io
    kind: VolumeSnapshot
    name: new-snapshot-demo
```

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: foo-pvc
  namespace: ns1
spec:
  storageClassName: example
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  dataSourceRef:
    apiGroup: snapshot.storage.k8s.io
```

```
    kind: VolumeSnapshot
    name: new-snapshot-demo
    namespace: default
  volumeMode: Filesystem
```

What's next

- Learn about Persistent Volumes.
- Learn about CSI Volume Cloning.
- Learn about Volume Snapshots.
- Read about the feature gates mentioned on this page.

12 - Storage Capacity

Storage capacity is limited and may vary depending on the node on which a pod runs: network-attached storage might not be accessible by all nodes, or storage is local to a node to begin with.

ⓘ **FEATURE STATE:** Kubernetes v1.24 [stable]

This page describes how Kubernetes keeps track of storage capacity and how the scheduler uses that information to schedule Pods onto nodes that have access to enough storage capacity for the remaining missing volumes. Without storage capacity tracking, the scheduler may choose a node that doesn't have enough capacity to provision a volume and multiple scheduling retries will be needed.

Before you begin

Kubernetes v1.36 includes cluster-level API support for storage capacity tracking. To use this you must also be using a CSI driver that supports capacity tracking. Consult the documentation for the CSI drivers that you use to find out whether this support is available and, if so, how to use it. If you are not running Kubernetes v1.36, check the documentation for that version of Kubernetes.

API

There are two API extensions for this feature:

- CSIStorageCapacity objects: these get produced by a CSI driver in the namespace where the driver is installed. Each object contains capacity information for one storage class and defines which nodes have access to that storage.
- The `CSIDriverSpec.StorageCapacity` field: when set to `true`, the Kubernetes scheduler will consider storage capacity for volumes that use the CSI driver.

Scheduling

Storage capacity information is used by the Kubernetes scheduler if:

- a Pod uses a volume that has not been created yet,
- that volume uses a StorageClass which references a CSI driver and uses `WaitForFirstConsumer` volume binding mode, and
- the `CSIDriver` object for the driver has `StorageCapacity` set to true.

In that case, the scheduler only considers nodes for the Pod which have enough storage available to them. This check is very simplistic and only compares the size of the volume against the capacity listed in `CSIStorageCapacity` objects with a topology that includes the node.

For volumes with `Immediate` volume binding mode, the storage driver decides where to create the volume, independently of Pods that will use the volume. The scheduler then schedules Pods onto nodes where the volume is available after the volume has been created.

For CSI ephemeral volumes, scheduling always happens without considering storage capacity. This is based on the assumption that this volume type is only used by special CSI drivers which are local to a node and do not need significant resources there.

Rescheduling

When a node has been selected for a Pod with `WaitForFirstConsumer` volumes, that decision is still tentative. The next step is that the CSI storage driver gets asked to create the volume with a hint that the volume is supposed to be available on the selected node.

Because Kubernetes might have chosen a node based on out-dated capacity information, it is possible that the volume cannot really be created. The node selection is then reset and the Kubernetes scheduler tries again to find a node for the Pod.

Limitations

Storage capacity tracking increases the chance that scheduling works on the first try, but cannot guarantee this because the scheduler has to decide based on potentially out-dated information. Usually, the same retry mechanism as for scheduling without any storage capacity information handles scheduling failures.

One situation where scheduling can fail permanently is when a Pod uses multiple volumes: one volume might have been created already in a topology segment which then does not have enough capacity left for another volume. Manual intervention is necessary

to recover from this, for example by increasing capacity or deleting the volume that was already created.

What's next

- For more information on the design, see the Storage Capacity Constraints for Pod Scheduling KEP.

13 - Node-specific Volume Limits

This page describes the maximum number of volumes that can be attached to a Node for various cloud providers.

Cloud providers like Google, Amazon, and Microsoft typically have a limit on how many volumes can be attached to a Node. It is important for Kubernetes to respect those limits. Otherwise, Pods scheduled on a Node could get stuck waiting for volumes to attach.

Kubernetes default limits

The Kubernetes scheduler has default limits on the number of volumes that can be attached to a Node:

Cloud service	Maximum volumes per Node
Amazon Elastic Block Store (EBS)	39
Google Persistent Disk	16
Microsoft Azure Disk Storage	16

Dynamic volume limits

ⓘ **FEATURE STATE:** Kubernetes v1.17 [stable]

Dynamic volume limits are supported for following volume types.

- Amazon EBS
- Google Persistent Disk
- Azure Disk
- CSI

For volumes managed by in-tree volume plugins, Kubernetes automatically determines the Node type and enforces the appropriate maximum number of volumes for the node. For example:

- On Google Compute Engine, up to 127 volumes can be attached to a node, depending on the node type.
- For Amazon EBS disks on M5,C5,R5,T3 and Z1D instance types, Kubernetes allows only 25 volumes to be attached to a Node. For other instance types on Amazon Elastic Compute Cloud (EC2), Kubernetes allows 39 volumes to be attached to a Node.
- On Azure, up to 64 disks can be attached to a node, depending on the node type. For more details, refer to Sizes for virtual machines in Azure.
- If a CSI storage driver advertises a maximum number of volumes for a Node (using `NodeGetInfo`), the kube-scheduler honors that limit. Refer to the CSI specifications for details.
- For volumes managed by in-tree plugins that have been migrated to a CSI driver, the maximum number of volumes will be the one reported by the CSI driver.

Mutable CSI Node Allocatable Count

ⓘ **FEATURE STATE:** Kubernetes v1.36 [stable](enabled by default)

CSI drivers can dynamically adjust the maximum number of volumes that can be attached to a Node at runtime. This enhances scheduling accuracy and reduces pod scheduling failures due to changes in resource availability.

To use this feature, you must enable the `MutableCSINodeAllocatableCount` feature gate on the following components:

- `kube-apiserver`
- `kubelet`

Periodic Updates

When enabled, CSI drivers can request periodic updates to their volume limits by setting the `nodeAllocatableUpdatePeriodSeconds` field in the `CSIDriver` specification. For example:

```
apiVersion: storage.k8s.io/v1
kind: CSIDriver
metadata:
```

```
  name: hostpath.csi.k8s.io
spec:
  nodeAllocatableUpdatePeriodSeconds: 60
```

Kubelet will periodically call the corresponding CSI driver's `NodeGetInfo` endpoint to refresh the maximum number of attachable volumes, using the interval specified in `nodeAllocatableUpdatePeriodSeconds`. The minimum allowed value for this field is 10 seconds.

If a volume attachment operation fails with a `ResourceExhausted` error (gRPC code 8), Kubernetes triggers an immediate update to the allocatable volume count for that Node. Additionally, kubelet marks affected pods as Failed, allowing their controllers to handle recreation. This prevents pods from getting stuck indefinitely in the `ContainerCreating` state.

Preventing Pod placement without CSI driver

> ⓘ **FEATURE STATE:** Kubernetes v1.35 [alpha](disabled by default)

If `VolumeLimitScaling` feature gate is enabled and a CSI driver has corresponding `CSIDriver` object installed with `spec.preventPodSchedulingIfMissing` set to true then scheduler will prevent pod placement to nodes that do not yet have CSI driver installed. For example:

```
apiVersion: storage.k8s.io/v1
kind: CSIDriver
metadata:
  name: hostpath.csi.k8s.io
spec:
  preventPodSchedulingIfMissing: true
```

This limitation only applies to pods that require corresponding CSI volume.

CSI volume attach limits and cluster autoscaler

If `--enable-csi-node-aware-scheduling` option is enabled in cluster-autoscaler, then cluster-autoscaler can accurately calculate number of nodes required to satisfy pending pods that require CSI volumes.

If you are using cluster-autoscaler in your Kubernetes cluster, we do not recommend preventing pod placement via `PreventPodSchedulingIfMissing` field, unless cluster-autoscaler also has `--enable-csi-node-aware-scheduling` command line option enabled. Underlying reason for this limitation while `VolumeLimitScaling` feature remains in alpha is - preventing pod placement can break scheduling simulation cluster-autoscaler runs if cluster-autoscaler is not already aware of CSI volume limits. We expect this limitation to go away once `--enable-csi-node-aware-scheduling` becomes enabled by default in cluster-autoscaler.

Command line `--enable-csi-node-aware-scheduling` in cluster-autoscaler can be enabled regardless of `VolumeLimitScaling` feature state in Kubernetes. We recommend enabling it if your cluster is using CSI volumes and you are running into issues related to, too many pods crowding a node when a new node is spun via cluster-autoscaler, because current version of cluster-autoscaler does not compute correct number of nodes required to satisfy all pending pods.

14 - Local ephemeral storage

Nodes have local ephemeral storage, backed by locally-attached writeable devices or, sometimes, by RAM. "Ephemeral" means that there is no long-term guarantee about durability.

Pods use ephemeral local storage for scratch space, caching, and for logs. The kubelet can provide scratch space to Pods using local ephemeral storage to mount `emptyDir` volumes into containers.

The kubelet also uses this kind of storage to hold node-level container logs, container images, and the writable layers of running containers.

> **Caution:**
> If a node fails, the data in its ephemeral storage can be lost. Your applications cannot expect any performance SLAs (disk IOPS for example) from local ephemeral storage.

> **Note:**
> To make the resource quota work on ephemeral-storage, two things need to be done:
>
> - An admin sets the resource quota for ephemeral-storage in a namespace.
> - A user needs to specify limits for the ephemeral-storage resource in the Pod spec.
>
> If the user doesn't specify the ephemeral-storage resource limit in the Pod spec, the resource quota is not enforced on ephemeral-storage.

Kubernetes lets you track, reserve and limit the amount of ephemeral local storage a Pod can consume.

Configurations for local ephemeral storage

Kubernetes supports the following ways to configure local ephemeral storage on a node:

Single filesystem | Runtime filesystem | Split image filesystem

In this configuration, you place all different kinds of ephemeral local data (`emptyDir` volumes, writeable layers, container images, logs) into one filesystem.

The kubelet also writes node-level container logs and treats these similarly to ephemeral local storage.

The kubelet writes logs to files inside its configured log directory (`/var/log` by default); and has a base directory for other locally stored data (`/var/lib/kubelet` by default).

Typically, both `/var/lib/kubelet` and `/var/log` are on the system root filesystem, and the kubelet is designed with that layout in mind.

Your node can have as many other filesystems, not used for Kubernetes, as you like.

The node-pressure eviction page refers to these observed filesystems as `nodefs` , `imagefs` , and `containerfs` . Those names do not always mean separate mount points.

The kubelet can measure local storage use when you set up the node using one of the supported configurations for local ephemeral storage.

If you have a different configuration, then the kubelet does not apply resource limits for ephemeral local storage.

> **Note:**
> The kubelet tracks `tmpfs` emptyDir volumes as container memory use, rather than as local ephemeral storage.

> **Note:**
> The kubelet can only track ephemeral storage on the filesystems it observes through the supported layouts. If you mount extra filesystems under paths such as `/var/lib/kubelet`, `/var/log`, or the container runtime storage directory outside those layouts, the kubelet might not report ephemeral storage correctly.

Setting requests and limits for local ephemeral storage

You can specify `ephemeral-storage` for managing local ephemeral storage. Each container of a Pod can specify either or both of the following:

- `spec.containers[].resources.limits.ephemeral-storage`
- `spec.containers[].resources.requests.ephemeral-storage`

Limits and requests for `ephemeral-storage` are measured in byte quantities. You can express storage as a plain integer or as a fixed-point number using one of these suffixes: E, P, T, G, M, k. You can also use the power-of-two equivalents: Ei, Pi, Ti, Gi, Mi, Ki. For example, the following quantities all represent roughly the same value:

- `128974848`
- `129e6`
- `129M`
- `123Mi`

Pay attention to the case of the suffixes. If you request `400m` of ephemeral-storage, this is a request for 0.4 bytes. Someone who types that probably meant to ask for 400 mebibytes (`400Mi`) or 400 megabytes (`400M`).

In the following example, the Pod has two containers. Each container has a request of 2GiB of local ephemeral storage. Each container has a limit of 4GiB of local ephemeral storage. Therefore, the Pod has a request of 4GiB of local ephemeral storage, and a limit of 8GiB of local ephemeral storage. 500Mi of that limit could be consumed by the `emptyDir` volume.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: frontend
spec:
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        ephemeral-storage: "2Gi"
      limits:
```

```
        ephemeral-storage: "4Gi"
    volumeMounts:
    - name: ephemeral
      mountPath: "/tmp"
  - name: log-aggregator
    image: images.my-company.example/log-aggregator:v6
    resources:
      requests:
        ephemeral-storage: "2Gi"
      limits:
        ephemeral-storage: "4Gi"
    volumeMounts:
    - name: ephemeral
      mountPath: "/tmp"
  volumes:
    - name: ephemeral
      emptyDir:
        sizeLimit: 500Mi
```

How Pods with ephemeral-storage requests are scheduled

When you create a Pod, the Kubernetes scheduler selects a node for the Pod to run on. Each node has a maximum amount of local ephemeral storage it can provide for Pods. For more information, see Node Allocatable.

The scheduler ensures that the sum of the resource requests of the scheduled containers is less than the capacity of the node.

Ephemeral storage consumption management

If the kubelet is managing local ephemeral storage as a resource, then the kubelet measures storage use in:

- `emptyDir` volumes, except *tmpfs* `emptyDir` volumes
- directories holding node-level logs
- writeable container layers

If a Pod is using more ephemeral storage than you allow it to, the kubelet sets an eviction signal that triggers Pod eviction.

For container-level isolation, if a container's writable layer and log usage exceeds its storage limit, the kubelet marks the Pod for eviction.

For pod-level isolation the kubelet works out an overall Pod storage limit by summing the limits for the containers in that Pod. In this case, if the sum of the local ephemeral storage usage from all containers and also the Pod's `emptyDir` volumes exceeds the overall Pod storage limit, then the kubelet also marks the Pod for eviction.

> **Caution:**
> If the kubelet is not measuring local ephemeral storage, then a Pod that exceeds its local storage limit will not be evicted for breaching local storage resource limits.
>
> However, if the filesystem space for writeable container layers, node-level logs, or `emptyDir` volumes falls low, the node taints itself as short on local storage and this taint triggers eviction for any Pods that don't specifically tolerate the taint.
>
> See the supported configurations for ephemeral local storage.

The kubelet supports different ways to measure Pod storage use:

Periodic scanning | Filesystem project quota

The kubelet performs regular, scheduled checks that scan each `emptyDir` volume, container log directory, and writeable container layer.

The scan measures how much space is used.

> **Note:**
> In this mode, the kubelet does not track open file descriptors for deleted files.
>
> If you (or a container) create a file inside an `emptyDir` volume, something then opens that file, and you delete the file while it is still open, then the inode for the

> deleted file stays until you close that file but the kubelet does not categorize the space as in use.

What's next

- Read about project quotas in XFS

15 - Volume Health Monitoring

FEATURE STATE: Kubernetes v1.21 [alpha]

CSI volume health monitoring allows CSI Drivers to detect abnormal volume conditions from the underlying storage systems and report them as events on PVCs or Pods.

Volume health monitoring

Kubernetes *volume health monitoring* is part of how Kubernetes implements the Container Storage Interface (CSI). Volume health monitoring feature is implemented in two components: an External Health Monitor controller, and the kubelet.

If a CSI Driver supports Volume Health Monitoring feature from the controller side, an event will be reported on the related PersistentVolumeClaim (PVC) when an abnormal volume condition is detected on a CSI volume.

The External Health Monitor controller also watches for node failure events. You can enable node failure monitoring by setting the `enable-node-watcher` flag to true. When the external health monitor detects a node failure event, the controller reports an Event will be reported on the PVC to indicate that pods using this PVC are on a failed node.

If a CSI Driver supports Volume Health Monitoring feature from the node side, an Event will be reported on every Pod using the PVC when an abnormal volume condition is detected on a CSI volume. In addition, Volume Health information is exposed as Kubelet VolumeStats metrics. A new metric kubelet_volume_stats_health_status_abnormal is added. This metric includes two labels: `namespace` and `persistentvolumeclaim`. The count is either 1 or 0. 1 indicates the volume is unhealthy, 0 indicates volume is healthy. For more information, please check KEP.

> **Note:**
> You need to enable the `CSIVolumeHealth` feature gate to use this feature from the node side.

What's next

See the CSI driver documentation to find out which CSI drivers have implemented this feature.

16 - Windows Storage

This page provides an storage overview specific to the Windows operating system.

Persistent storage

Windows has a layered filesystem driver to mount container layers and create a copy filesystem based on NTFS. All file paths in the container are resolved only within the context of that container.

- With Docker, volume mounts can only target a directory in the container, and not an individual file. This limitation does not apply to containerd.
- Volume mounts cannot project files or directories back to the host filesystem.
- Read-only filesystems are not supported because write access is always required for the Windows registry and SAM database. However, read-only volumes are supported.
- Volume user-masks and permissions are not available. Because the SAM is not shared between the host & container, there's no mapping between them. All permissions are resolved within the context of the container.

As a result, the following storage functionality is not supported on Windows nodes:

- Volume subpath mounts: only the entire volume can be mounted in a Windows container
- Subpath volume mounting for Secrets
- Host mount projection
- Read-only root filesystem (mapped volumes still support `readOnly`)
- Block device mapping
- Memory as the storage medium (for example, `emptyDir.medium` set to `Memory`)
- File system features like uid/gid; per-user Linux filesystem permissions
- Setting secret permissions with DefaultMode (due to UID/GID dependency)
- NFS based storage/volume support
- Expanding the mounted volume (resizefs)

Kubernetes volumes enable complex applications, with data persistence and Pod volume sharing requirements, to be deployed on Kubernetes. Management of persistent volumes associated with a specific storage back-end or protocol includes actions such as provisioning/de-provisioning/resizing of volumes, attaching/detaching a volume to/from a

Kubernetes node and mounting/dismounting a volume to/from individual containers in a pod that needs to persist data.

Volume management components are shipped as Kubernetes volume plugin. The following broad classes of Kubernetes volume plugins are supported on Windows:

- `FlexVolume plugins`
  - Please note that FlexVolumes have been deprecated as of 1.23
- `CSI Plugins`

In-tree volume plugins

The following in-tree plugins support persistent storage on Windows nodes:

- `azureFile`
- `vsphereVolume`